



第2章 运算方法和运算器I





2.1 数据与文字的表达方法





数据编码与表示

■ 计算机中的数据

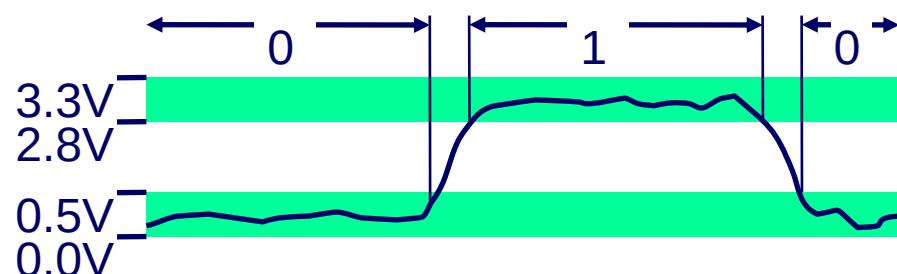
- ◆ 整数、浮点数、字符（串）、逻辑值
- ◆ 需要编码进行表示

■ 编码原则

- ◆ 使用少量简单的基本符号
- ◆ 一定的规则
- ◆ 表示大量复杂的信息

■ 二进制码0、1

- ◆ 符号个数最少，物理上容易实现
- ◆ 与二值逻辑的“真”、“假”两个值对应
- ◆ 用二进制码表示数值数据运算规则简单





无符号数和有符号数

- 无符号数指的是不带符号位的数，例如：
1个16位二进制数，表示范围为0 ~ 65535
在C语言中，用**unsigned short int**类型进行声明
- 有符号数指的是带符号位的数，最左边的位用作符号位，用“0”表示正，“1”表示负，例如
有符号数+0001001 00000000，表示为0x0900
有符号数-0001001 00000000，表示为0x8900
在C语言中，用**short int**类型进行声明

注：在64位机上，int类型用32位表示





计算机中常用的数据表示

- 定点表示：数值范围有限，处理简单
 - ◆ 机器中所有数据的小数点位置固定不变
 - ◆ 不使用记号 “.” 来表示小数点
 - ◆ 定点数可以表示纯小数或纯整数

- 浮点表示：数值范围很大，但处理过程复杂

- 十进制表示
 - ◆ 非压缩BCD
 - ◆ 压缩BCD





定点数的表示方法

■ 纯小数

x_0	$x_1 \ x_2 \ \dots \ x_{n-1} \ x_n$
-------	-------------------------------------

- ◆ x_0 表示数的符号，数值0和1分别表示正号和负号，其余为代表数的量值
- ◆ 小数点位于 x_0 和 x_1 之间.
- ◆ 表示范围为 $0 \leq |x| \leq 1 - 2^{-n}$

■ 纯整数

x_0	$x_1 \ x_2 \ \dots \ x_{n-1} \ x_n$
-------	-------------------------------------

- ◆ x_0 表示数的符号，数值0和1分别表示正号和负号，其余为代表数的量值
- ◆ 小数点位于最低位 x_n 的右边
- ◆ 表示范围为 $0 \leq |x| \leq 2^n - 1$





数的机器码表示

- 在计算机中将数的符号位和数值位一起编码，可以方便数的运算操作
 - 为了区别有符号数和机器中把符号“数字化”的数，通常将前者称为**真值**，后者称为**机器数或机器码**
 - 常用的机器码包括
 - ◆ 原码
 - ◆ 补码
 - ◆ 反码
 - ◆ 移码
- } **有符号数**





原码表示法 (1)

■ 定点小数

x_0	x_1	x_2	$\dots$	x_{n-1}	x_n
-------	-------	-------	---------	-----------	-------

◆ 定义

$$\begin{cases}
 x & 1 > x \geq 0 \\
 1 - x = 1 + |x| & 0 \geq x > -1
 \end{cases}$$

◆ 举例

$$x = +0.1001 \longrightarrow [x]_{\text{原}} = 0.1001$$

$$x = -0.1001 \longrightarrow [x]_{\text{原}} = 1.1001$$

真值

机器码





原码表示法 (2)

■ 定点整数

x_0	x_1	x_2	$\dots$	x_{n-1}	x_n
-------	-------	-------	---------	-----------	-------

◆ 定义

$$[x]_{\text{原}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^n - x = 2^n + |x| & 0 \geq x > -2^n \end{cases}$$

◆ 举例

$$x = +0111001 \longrightarrow [x]_{\text{原}} = 00111001$$

$$x = -0111001 \longrightarrow [x]_{\text{原}} = 10111001$$

真值

机器码





原码表示法 (3)

■ 0的原码

◆ $[+0]_{\text{原}} = 00000000$

◆ $[-0]_{\text{原}} = 10000000$

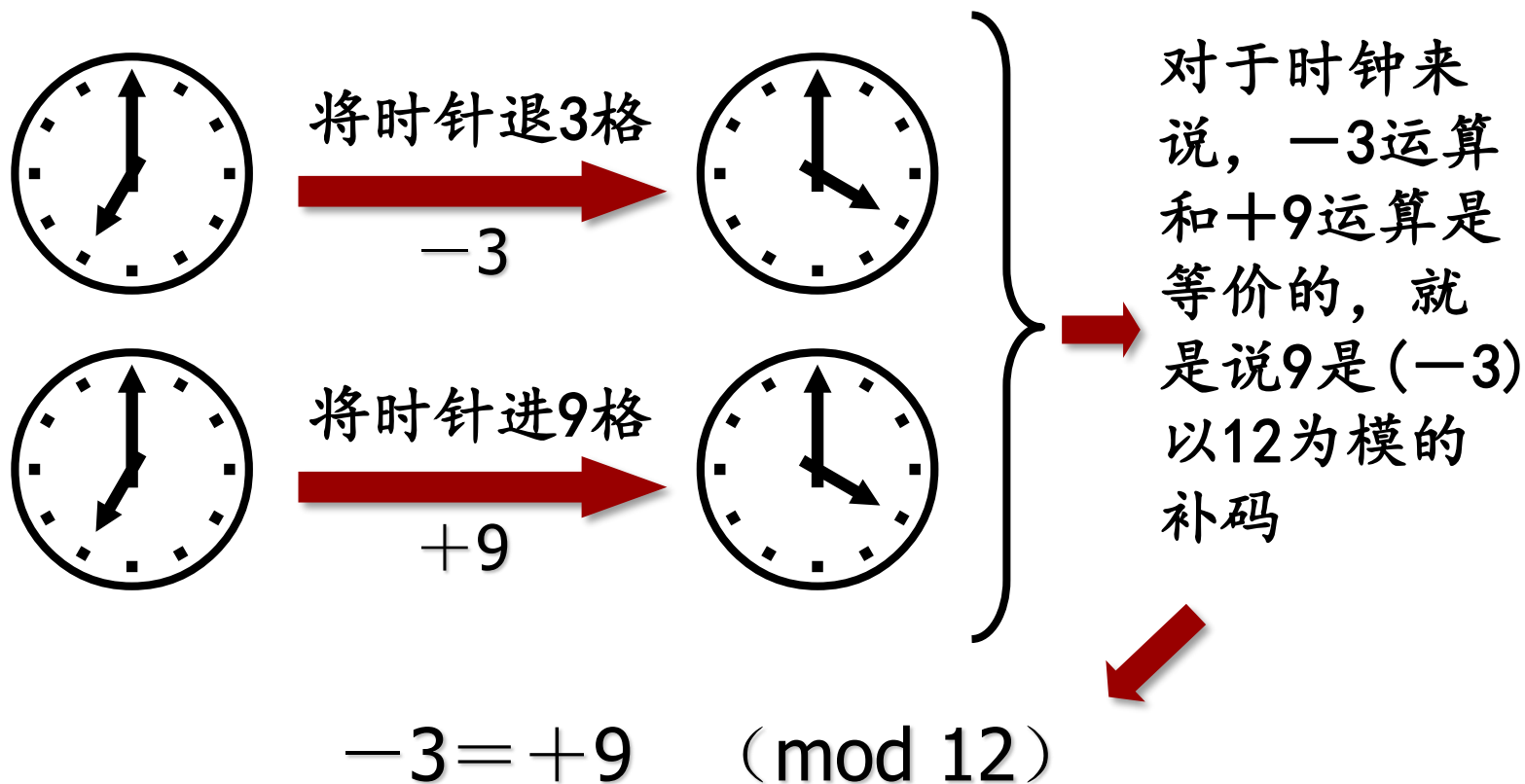
- 正数的原码符号位为0，数值位不变；负数的原码符号位为1，数值位不变
- 采用原码表示法简单明了，易于和真值转换，但加法运算复杂





补码表示法 (1)

将指针式时钟从7点钟调到4点钟，有两种做法：



减法可转换为加法！





补码表示法 (2)

■ 定点小数

x_0	x_1	x_2	$\dots$	x_{n-1}	x_n
-------	-------	-------	---------	-----------	-------

◆ 定义

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x = 2 - |x| & 0 \geq x \geq -1 \end{cases} \quad (\text{mod } 2)$$

◆ 举例

$$x = +0.1011 \quad \longrightarrow \quad [x]_{\text{补}} = 0.1011$$

$$x = -0.1011 \quad \longrightarrow$$

$$[x]_{\text{补}} = 10 + x = 10 - 0.1011 = 1.0101$$

补码: 2's Complement





补码表示法 (3)

■ 定点整数

x_0	x_1	x_2	$\dots$	x_{n-1}	x_n
-------	-------	-------	---------	-----------	-------

◆ 定义

$$[x]_{\text{补}} = \begin{cases} x & 2^n > x \geq 0 \\ 2^{n+1} + x = 2^{n+1} - |x| & 0 \geq x \geq -2^n \end{cases} \pmod{2^{n+1}}$$

◆ 举例

$$x = +0111001 \rightarrow [x]_{\text{补}} = 00111001$$

$$x = -0111001 \rightarrow$$

$$[x]_{\text{补}} = \underset{2^{7+1}}{100000000} + x = 100000000 - 0111001 = 11000111$$

回顾CSAPP中Two's Complement公式

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$





补码表示法 (4)

■ 0的补码

$$[+0]_{\text{补}} = [-0]_{\text{补}} = 00000000 \pmod{2^{n+1}}$$

- 采用补码进行减法运算比原码方便，**减法运算可变成加法运算**





反码表示法 (1)

■ 定点小数

x_0	x_1	x_2	$\dots$	x_{n-1}	x_n
-------	-------	-------	---------	-----------	-------

◆ 定义

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2 - 2^{-n}) + x & 0 \geq x > -1 \end{cases}$$

◆ 举例

$$x = +0.1011 \quad \longrightarrow \quad [x]_{\text{反}} = 0.1011$$

$$x = -0.1011 \quad \longrightarrow$$

$$[x]_{\text{反}} = 10 - 0.0001 - 0.1011 = 1.1111 - 0.1011 = 1.0100$$





反码表示法 (2)

■ 定点小数

◆ 对比负数求补码和反码公式

$$[x]_{\text{反}} = (2 - 2^{-n}) + x$$

$$[x]_{\text{补}} = 2 + x = [x]_{\text{反}} + 2^{-n}$$

- ◆ 结论：对一个负数求补码，其方法是将符号位置为1，其余各位求反，然后在末位上加1。即：负数的补码等于其反码加1

■ 0的反码

- ◆ $[+0]_{\text{反}} = 00000000$

- ◆ $[-0]_{\text{反}} = 11111111$





反码表示法 (3)

■ 定点整数

x_0	x_1	x_2	$\dots$	x_{n-1}	x_n
-------	-------	-------	---------	-----------	-------

◆ 定义

$$[x]_{\text{反}} = \begin{cases} x & 2^n > x \geq 0 \\ (2^{n+1} - 1) + x & 0 \geq x > -2^n \end{cases}$$

◆ 举例

$$x = +0111001 \longrightarrow [x]_{\text{反}} = 00111001$$

$$x = -0111001 \longrightarrow$$

$$[x]_{\text{反}} = 11111111 + x = 11111111 - 0111001 = 11000110$$

若 x 为负数，将符号位置1，其余各位按位取反

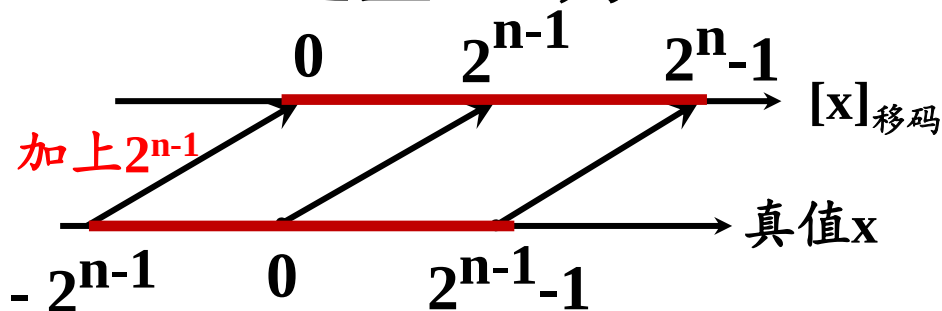




移码表示法

- 移码是将每一个数值加上一个偏移值bias
- 当编码位数为n位时，移码的定义为：

$$[x]_{\text{移}} = x + 2^{n-1} \quad -2^{n-1} \leq x < 2^{n-1} \quad \text{这里 } 2^{n-1} \text{ 为 bias}$$



- 举例（假设移码为4位）

$$x = -7 \longrightarrow [x]_{\text{移}} = x + 2^{4-1} = -7 + 8 = 0001$$

$$x = +5 \longrightarrow [x]_{\text{移}} = x + 2^{4-1} = 5 + 8 = 1101$$

x	移码
-8	0000
-7	0001
0	1000
+1	1001
+6	1110
+7	1111

移码可用来表示浮点数的指数
移码和补码尾数相同，符号位相反





8位无符号数和有符号数的表示

考虑 8 位二进制数（其中 1 位可用做符号位），用其分别代表无符号数、原码、补码和反码，对应的真值范围如下表

二进制编码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	± 0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0





动手做

1、若某数 x 的真值为 -0.1010 ，在计算机中该数表示为 1.0110 ，则该数所用的编码方法是

A.原码 B.补码 C.反码 D.移码





机器码表示小结

- 原、反、补和移码4种机器码表示是对**有符号数**而言
- 若 x 为正数，则 $[x]_{\text{原}} = [x]_{\text{反}} = [x]_{\text{补}} = x$
- 最高位为符号位
 - ◆ $[x]_{\text{原}}$ 、 $[x]_{\text{反}}$ 、 $[x]_{\text{补}}$ 用“0”表示正号，用“1”表示负号；
 - ◆ $[x]_{\text{移}}$ 用“1”表示正号，用“0”表示负号（也可认为没有符号位）
- $[0]_{\text{补}}$ 、 $[0]_{\text{移}}$ 有唯一编码， $[0]_{\text{原}}$ 、 $[0]_{\text{反}}$ 有两种编码
- 移码与补码的尾码相同，而符号位相反
- 补码、反码和移码的符号位在加减运算时**视为数值**，而原码的符号位必须单独处理





例题

设机器字长16位，定点表示，尾数15位，数符1位，问：

- (1) 定点原码整数表示时，最大正数、最小负数各是多少？
- (2) 定点原码小数表示时，最大正数、最小负数各是多少？
- (3) 定点补码整数表示时，最大正数、最小负数各是多少？

(1) 定点原码整数表示

最大正数值 = **0**111 1111 1111 1111 = $(2^{15}-1)_{10} = (+32767)_{10}$

最小负数值 = **1**111 1111 1111 1111 = $-(2^{15}-1)_{10} = (-32767)_{10}$

(2) 定点原码小数表示

最大正数值 = **0**111 1111 1111 1111 = $(1-2^{-15})_{10} = (0.111...11)_2$

最小负数值 = **1**111 1111 1111 1111 = $-(1-2^{-15})_{10} = (-0.111..11)_2$

(3) 定点补码整数表示

最大正数值 = **0**111 1111 1111 1111 = $(2^{15}-1)_{10} = (+32767)_{10}$

最小负数值 = **1**000 0000 0000 0000 = $(-2^{15}+0)_{10} = (-32768)_{10}$





64位现代计算机中

C语言

```
int x = 100;
```

```
int y = -100;
```

编译

机器数

0x00000064

0xFFFFFFFF9C

补码

编译

```
unsigned short int x=65534;    0xFFFE
```

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

现代计算机中有符号数都是用补码表示的





典型计算机的数据表示

C Data Type	Typical 32-bit	Intel IA32	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	4	8
long long	8	8	8
float	4	4	4
double	8	8	8
long double	8	10/12	10/16
pointer	4	4	8





十进制数表示方法

- 字符串形式：一个字节存放一个十进制的数位或符号位
- 压缩的十进制数：一个字节存放两个十进制的数位
 - ◆ 用四位二进制表示一位十进制，16个编码状态选用其中的10个编码
 - ◆ 有多种方案
 - 8421码
 - 余3码
 - 循环码





8421 余3码 循环码

0	0000	0011	0000
1	0001	0100	0001
2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	1110
6	0110	1001	1010
7	0111	1010	1000
8	1000	1011	1100
9	1001	1100	0100

■ 有权码
■ 无权码





字符型数据的表示

字符作为人-机联系的媒介，是最重要的数据类型之一，最常见的是ASCII字符编码，每个字符用7位二进制数表示，可表示 128 个字符。在计算机内部用 1 个字节表示字符，最高位为0或用作奇偶校验位。

通用的两个标准字符集为：

■ ASCII 码

- ◆ 大小写字母、数字、专用符号、控制字符

■ Unicode或 ISO 10646

- ◆ 统一码，表示多种语言文字





Unicode编码

- **Unicode**是国际组织制定的可以容纳世界上所有文字和符号的字符编码方案，它为每种语言中的每个字符设定了唯一的二进制编码，以满足跨语言、跨平台进行文本转换、处理的要求
- **Unicode** 规划的码空间是17个平面（平面0至16），0x0000 至 0x10 FFFF。每个平面有 65536 个编码（0x0000-0xFFFF）
 - ◆ 平面2的43253个字符都是汉字
 - ◆ 平面0上定义了27973个汉字
- 在Unicode中，汉字的“汉”，其编码是0x6C49
- **UTF**（Unicode Transformation Format）是Unicode字符集转换格式，是一种储存和传送的格式。可理解为Unicode的压缩形式。UTF格式包括UTF-8、UTF-16、UTF-32
- **UTF-8**是一种变长字符编码，1~4个字节





例：UTF-8

- UTF-8以字节为单位对Unicode进行编码。从Unicode到UTF-8的编码方式如下：

		Unicode编码(十六进制)	UTF-8 字节流(二进制)
平面0 {		<u>000000</u> - <u>00007F</u>	0xxxxxxx
		<u>000080</u> - <u>0007FF</u>	110xxxxx 10xxxxxx
		<u>000800</u> - <u>00FFFF</u>	1110xxxx 10xxxxxx 10xxxxxx
		<u>010000</u> - <u>10FFFF</u>	11110xxx10xxxxxx10xxxxxx10xxxxxx

- “汉”字的Unicode编码是0x6C49
- 0x6C49 → 0110 1100 01 00 1001
- UTF-8为11100110 10110001 10001001，即E6 B1 89





汉字编码

计算机处理汉字信息的过程：

1. 汉字输入到计算机 – 汉字输入码
2. 计算机内部的表示和存储 – 汉字内码
3. 计算机向外部显示和打印 – 汉字字形（字模）码





汉字的表示方法（1）

■ 汉字交换码（国标码）

- ◆ 信息交换用汉字编码字符集，是我国1980年发布，1981年5月1日开始实施的一套国家标准，标准号是GB 2312—1980。
- ◆ 基本集共收入汉字6763个和非汉字图形字符682个
- ◆ 整个字符集分成94个区，每区有94个位。每个区位上只有一个字符，因此用汉字所在的区号（第1个字节表示）和位号（第2个字节表示）来进行编码，称为**区位码**。编码是唯一的，不会有重码字。
- ◆ 把换算成十六进制的区位码加上2020H，就得到**国标码**

■ 随后出现GBK字符集、GB18030字符集，扩充了字符集





汉字的表示方法（2）

■ 汉字输入码（外码）

◆ 为了通过键盘字符把汉字输入计算机而设计的一种编码。

汉字输入方案有很多，大致可分为以下4种类型

- 音码：如全拼、双拼、微软拼音等
- 形码：如五笔字型、郑码、表形码等
- 音形码：如智能ABC、自然码等
- 数字码：如区位码、电报码等
- 手写输入





汉字的表示方法（3）

■ 汉字机内码（内码）

- ◆ 用于在计算机内部存储汉字而设计的一种2字节编码。为了避免ASCII码和国标码一起使用时产生二义性问题，把国标码每个字节的最高位置1，用以区别7位ASCII码，称为汉字机内码

汉字机内码、国标码和区位码三者之间的关系为：

区位码（十六进制）的两个字节分别加20H得到对应的国标码；

汉字交换码（国标码）的两个字节分别加80H得到对应的机内码；

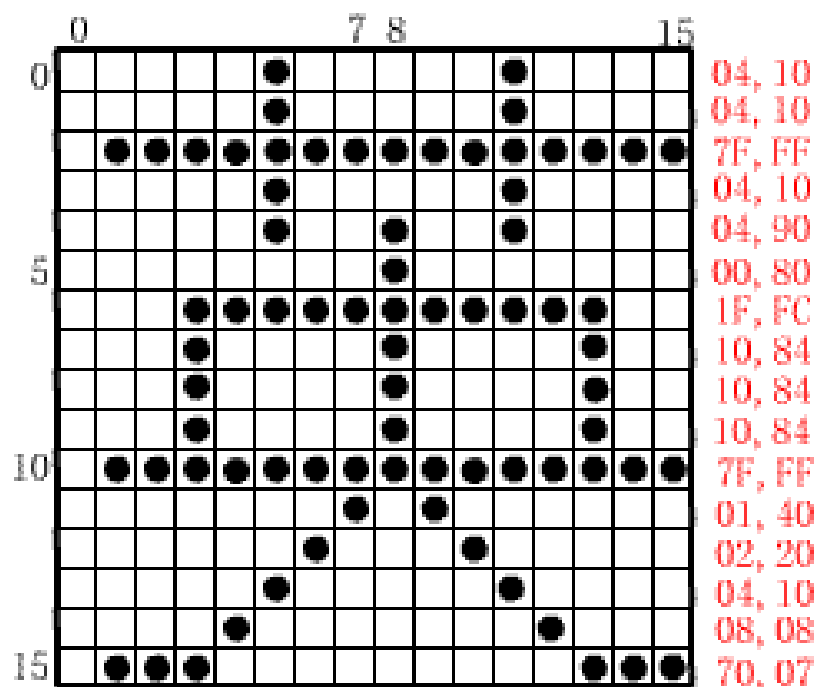




汉字的表示方法（4）

■ 汉字字模码（输出码）

- ◆ 用于把机内码转成能显示和打印的一种汉字编码，即用点阵表示的汉字代码





汉字表示方法小结

- 汉字的输入码、汉字内码、字模码是计算机中用于输入、内部处理、输出三种不同用途的编码
- 输入汉字时，实际上是通过输入法将输入编码转换为汉字内码
- 显示和打印汉字时，需要先将通过汉字内码在字库文件中找到对应的字模码





校验码

- 在数据存储、传输过程中，附加在数据中的可以用来检查和纠正因元件故障、噪声干扰等因素导致数据错误的编码
- 数据校验原理

信息位 (k 位)

校验码 (r 位)

- 分类
 - ◆ 检错码，如奇偶校验码、海明码、循环冗余校验码
 - ◆ 纠错码，如海明码、BCH码、RS码





奇偶校验码

设 $X = (x_0 x_1 \dots x_{n-1})$

■ 奇校验

- ◆ 加上检验位后，传输的位组中，“1”的个数一定为奇数；否则就发生了错误
- ◆ 奇校验位 $\overline{C} = x_0 \oplus x_1 \oplus \dots \oplus x_{n-1}$

■ 偶校验

- ◆ 加上检验位后，传输的位组中，“1”的个数一定为偶数；否则就发生了错误
- ◆ 偶校验位 $C = x_0 \oplus x_1 \oplus \dots \oplus x_{n-1}$

■ 奇偶校验能检测奇数个错误，无法检测偶数个错误





实例

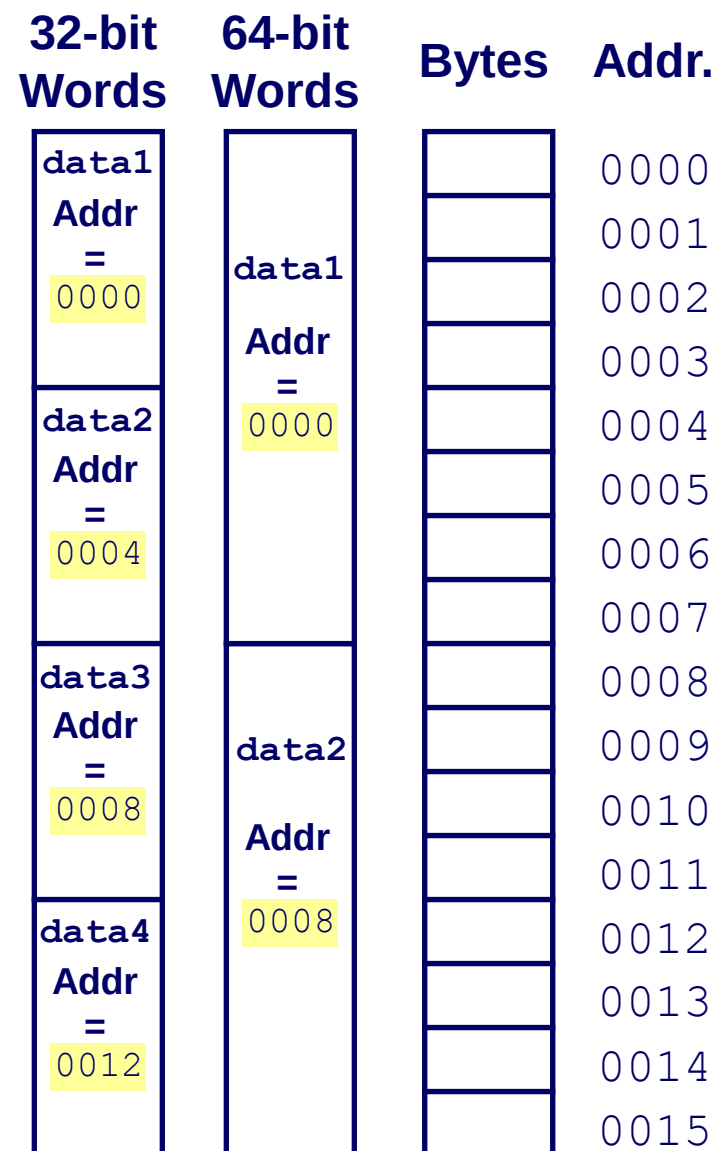
数据	奇校验编码	偶校验编码
10101010	10101010 1	10101010 0
01010100	01010100 0	01010100 1
00000000	00000000 1	00000000 0
11111111	11111111 1	11111111 0





数据在存储器中的地址

- 按字节寻址的存储器可视为一个很大的字节数组，数组下标为存储器地址，数组元素的值即为存储器存储的内容
- 多字节对象的地址就是对象中第一个字节的地址
- 连续存放的32位或64位的字数据的地址差值为4或8





存储器中的字节序

■ 小端方式Little Endian

- ◆ 低字节存放在小地址处，即低字节在前高字节后
- ◆ x86处理器

■ 大端方式Big Endian

- ◆ 低字节存放在大地址处，即高字节在前低字节后
- ◆ SPARC处理器，IBM Power处理器

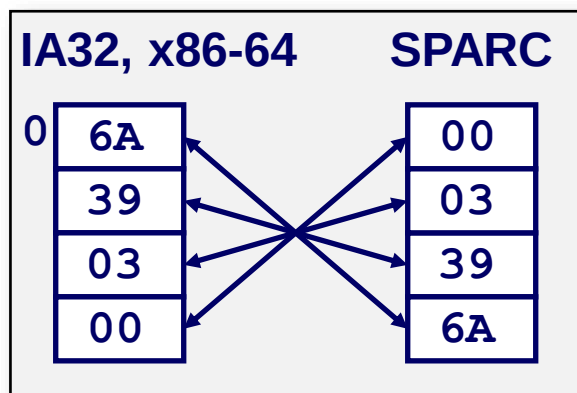




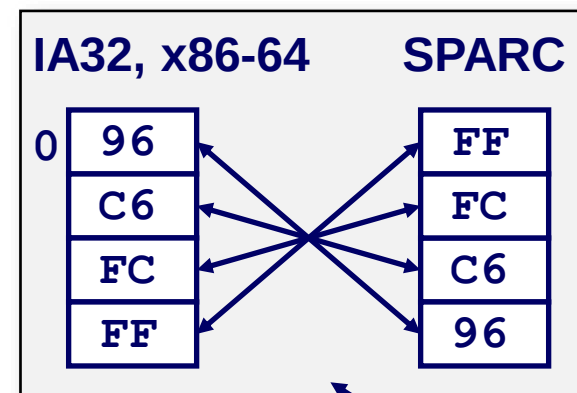
例 不同处理器存储整数的字节序

十进制:	211306				
二进制:	0011	0011	1001	0110	1010
十六进制:	3	3	9	6	A

int A = 211306;



int B = -211306;



二进制补码





概念回顾

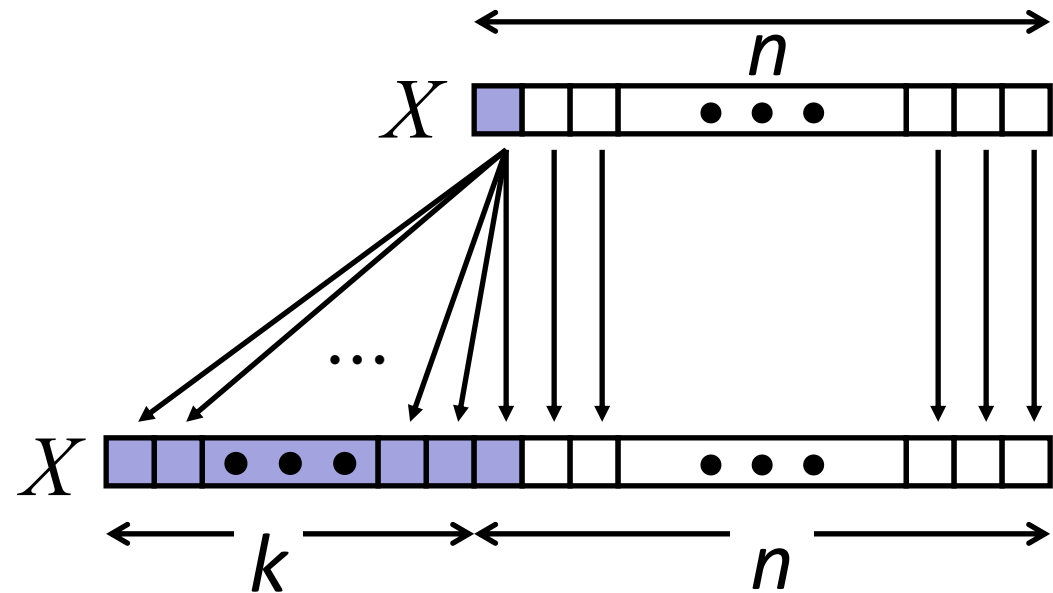
- 位扩展
- 位截断
- 移位操作





符号位扩展

- 将 n 位的有符号整数 x 转换为 $n+k$ 位，并保持值不变



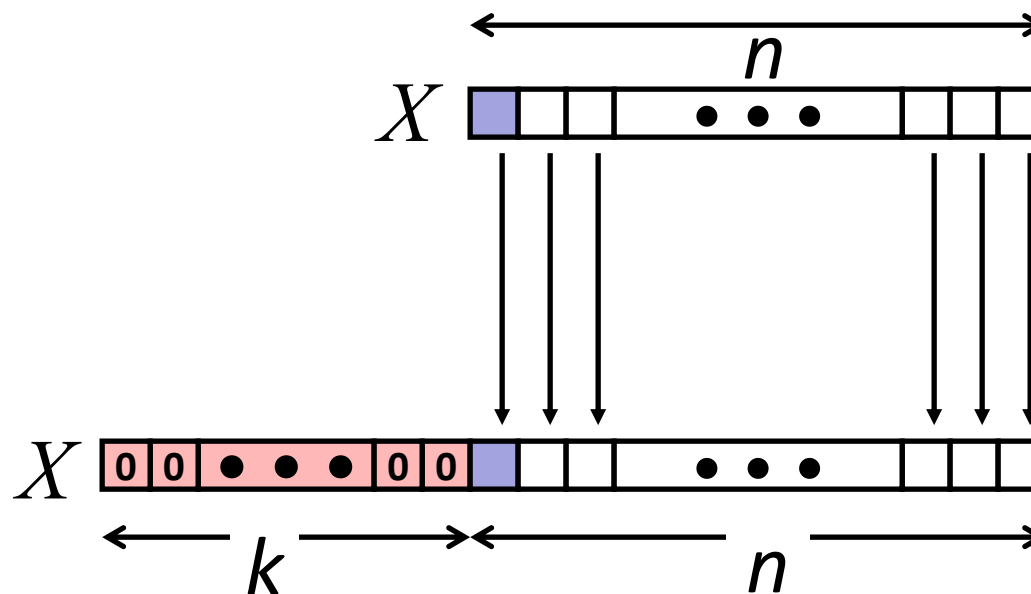
```
short int x = 15213;
int      ix = (int) x;
short int y = -15213;
int      iy = (int) y;
```

	Decimal	Hex	Binary
x	15213	3B 6D	00111011 01101101
ix	15213	00 00 3B 6D	00000000 00000000 00111011 01101101
y	-15213	C4 93	11000100 10010011
iy	-15213	FF FF C4 93	11111111 11111111 11000100 10010011



零扩展

- 将 n 位的无符号整数 x 转换为 $n+k$ 位，并保持值不变





动手做

在C语言中，`int`型占32位，`short`型占16位，若有下列语句

```
unsigned short x = 65530;
```

```
unsigned int y = x;
```

则执行后，`y`的十六进制表示为

A. 0000 7FFA

B. 0000 FFFA

C. FFFF 7FFA

D. FFFF FFFA





截断

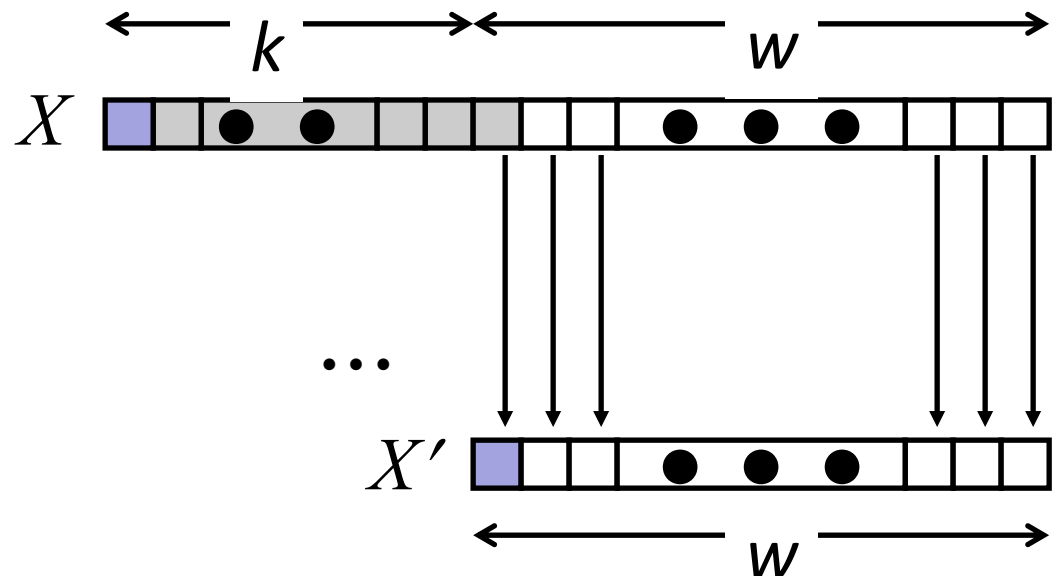
■ 任务:

- ◆ 给定 $k+w$ 位有符号或无符号整数 X
- ◆ 将其转换为 w 位整数 X' ，且对于“足够小” X 而言截断后的值保持不变

■ 规则:

- ◆ 删除前 k 位:

- ◆ $X' = x_{w-1}, x_{w-2}, \dots, x_0$





移位操作

- 机器中的移位操作实际上是一种运算
 - ◆ 左移1位 绝对值扩大，即： $\times 2$
 - ◆ 右移1位 绝对值缩小，即： $\div 2$
- 在计算机中，移位与加减配合，能够实现乘除运算
- 逻辑移位
 - ◆ 左移时，高位移出，低位补0
 - ◆ 右移时，低位移出，高位补0
- 算术移位（有符号数移位）
 - ◆ 左移时，高位移出，低位补0（若为负数反码，则补1）
 - ◆ 右移时
 - 原码：符号位固定不变，右移空出的数值位补0
 - 补码和反码：符号位固定不变，右移空出的数值位补符号位





算术移位操作举例（1）

- 例：机器字长为8位， $x = -20$ ，对其进行左移位和右移位，结果如下：

$[x]_{\text{原}} = 10010100$

左移： $[x]_{\text{原}} = 00101000$ + 40

右移： $[x]_{\text{原}} = 10001010$ - 10

$[x]_{\text{补}} = 11101100$

左移： $[x]_{\text{补}} = 11011000$

$[x]_{\text{原}} = 10101000$ - 40

右移： $[x]_{\text{补}} = 11110110$

$[x]_{\text{原}} = 10001010$ - 10

高位移出
原码左移补0

符号位不动
原码右移补0
低位移出

高位移出
补码左移补0

符号位不动
补码右移补
符号位，低
位移出





算术移位操作举例 (2)

$[x]_{\text{反}} = 11101011$

左移: $[x]_{\text{反}} = 11010111$

$[x]_{\text{原}} = 10101000 \quad - 40$

右移: $[x]_{\text{反}} = 11110101$

$[x]_{\text{原}} = 10001010 \quad - 10$

高位移出，负
数反码左移补1

符号位不动反
码右移补符号
位，低位移出





2.2 定点加法、减法运算





加法与减法运算

■ 补码加法公式

整数 $[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}} \pmod{2^{n+1}}$

小数 $[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}} \pmod{2}$

■ 补码减法公式

整数 $[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} = [x-y]_{\text{补}} \pmod{2^{n+1}}$

小数 $[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} = [x-y]_{\text{补}} \pmod{2}$





证明 $[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}}$ (1)

■ 假设 $|x| < 1$, $|y| < 1$, $|x + y| < 1$

■ 现分四种情况来证明

(1) $x > 0$, $y > 0$, 则 $x + y > 0$

$$[x]_{\text{补}} = x, [y]_{\text{补}} = y, [x + y]_{\text{补}} = x + y$$

所以等式成立.

(2) $x > 0$, $y < 0$, 则 $x + y > 0$ 或 $x + y < 0$

$$[x]_{\text{补}} = x, [y]_{\text{补}} = 2 + y,$$

$$[x]_{\text{补}} + [y]_{\text{补}} = x + 2 + y$$

若 $x + y > 0$, $2 + (x + y) > 2$, 进位2必丢失, 又因 $(x + y) > 0$ 故 $[x]_{\text{补}} + [y]_{\text{补}} = x + y = [x + y]_{\text{补}}$

若 $x + y < 0$, $2 + (x + y) < 2$, 又因 $(x + y) < 0$,

$$\text{故 } [x]_{\text{补}} + [y]_{\text{补}} = 2 + (x + y) = [x + y]_{\text{补}}$$

所以上式成立





证明 $[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}}$ (2)

(3) $x < 0$, $y > 0$, 则 $x + y > 0$ 或 $x + y < 0$

这种情况和第2种情况一样, 把 x 和 y 的位置对调即得证

(4) $x < 0$, $y < 0$, 则 $x + y < 0$

相加两数都是负数, 则其和也一定是负数

$$\because [x]_{\text{补}} = 2 + x, \quad [y]_{\text{补}} = 2 + y$$

$$\therefore [x]_{\text{补}} + [y]_{\text{补}} = 2 + x + 2 + y = \textcolor{red}{2} + (2 + x + y)$$

上式右边分为“2”和 $(2 + x + y)$ 两部分。既然 $(x + y)$ 是负数, 而其绝对值又小于1, 那么 $(2 + x + y)$ 就一定是小于2而大于1的数, 进位“ $\textcolor{red}{2}$ ”必丢失。又因 $(x + y) < 0$, 所以 $[x]_{\text{补}} + [y]_{\text{补}} = 2 + (x + y) = [x + y]_{\text{补}}$





举例

■ $x = 0.1001$, $y = 0.0101$, 求 $x + y$

解: $[x]_{\text{补}} = 0.1001$, $[y]_{\text{补}} = 0.0101$

$$\begin{array}{r}
 [x]_{\text{补}} \quad 0.1001 \\
 + [y]_{\text{补}} \quad 0.0101 \\
 \hline
 [x + y]_{\text{补}} \quad 0.1110
 \end{array}$$

所以 $x + y = +0.1110$





举例

■ $x = +0.1011$, $y = -0.0101$, 求 $x + y$

解: $[x]_{\text{补}} = 0.1011$, $[y]_{\text{补}} = 1.1011$

$$\begin{array}{r}
 [x]_{\text{补}} \quad 0.1011 \\
 + [y]_{\text{补}} \quad 1.1011 \\
 \hline
 [x + y]_{\text{补}} \quad 10.0110
 \end{array}$$

所以 $x + y = +0.0110$

补码加法：符号位要作为数的一部分参与运算，进行模2的加法





证明 $[x]_{\text{补}} - [y]_{\text{补}} = [x - y]_{\text{补}}$

补码减法的公式

$$[x]_{\text{补}} - [y]_{\text{补}} \stackrel{?}{=} [x - y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \quad (\text{mod } 2)$$

证明上述公式成立的关键是 $[-y]_{\text{补}} = -[y]_{\text{补}}$





证明 $[-y]_{\text{补}} = -[y]_{\text{补}}$

证：

令 $x = -y$ ，于是

$$[x + y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} = [-y]_{\text{补}} + [y]_{\text{补}}$$

又

$$[x + y]_{\text{补}} = [-y + y]_{\text{补}} = [0]_{\text{补}} = 0$$

故

$$[-y]_{\text{补}} + [y]_{\text{补}} = 0$$

也即

$$[-y]_{\text{补}} = -[y]_{\text{补}}$$





$[-y]_{\text{补}}$ 求补法则

■ 补码减法

$$[x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} = [x-y]_{\text{补}}$$

- 在进行补码减法前，需直接从 $[y]_{\text{补}}$ 求出 $[-y]_{\text{补}}$ 。最简单的方法是：对 $[y]_{\text{补}}$ 各位（包括符号位）“各位求反且末位加1”，即可得到 $[-y]_{\text{补}}$
- 写成运算表达式，则为 $[-y]_{\text{补}} = \neg [y]_{\text{补}} + 2^{-n}$
符号“ $\neg$ ”表示对 $[y]_{\text{补}}$ 做包括符号位在内的求反操作





举例

$x_1 = -0.1110$, $x_2 = 0.1101$, 求 $[x_1]_{\text{补}}$, $[-x_1]_{\text{补}}$,
 $[x_2]_{\text{补}}$, $[-x_2]_{\text{补}}$

【解:】 $[x_1]_{\text{补}} = 1.0010$

$$\begin{aligned} [-x_1]_{\text{补}} &= \rightarrow [x_1]_{\text{补}} + 2^{-4} \\ &= 0.1101 + 0.0001 \\ &= 0.1110 \end{aligned}$$

$$[x_2]_{\text{补}} = 0.1101$$

$$\begin{aligned} [-x_2]_{\text{补}} &= \rightarrow [x_2]_{\text{补}} + 2^{-4} \\ &= 1.0010 + 0.0001 \\ &= 1.0011 \end{aligned}$$





举例

$x = +0.1101$, $y = +0.0110$, 求 $x - y$

解: $[x]_{\text{补}} = 0.1101$

$[y]_{\text{补}} = 0.0110$, $[-y]_{\text{补}} = 1.1010$

$$\begin{array}{r}
 [x]_{\text{补}} \quad 0.1101 \\
 + [-y]_{\text{补}} \quad 1.1010 \\
 \hline
 [x - y]_{\text{补}} \quad 10.0111
 \end{array}$$

所以 $x - y = +0.0111$

在模2的意义下相加，即超过2的进位要丢掉





有符号整数加法运算例子

$x = +1011$, $y = +1001$, 求 $x + y$?

解: $[x]_{\text{补}} = 01011$, $[y]_{\text{补}} = 01001$

$$\begin{array}{r}
 [x]_{\text{补}} \quad 01011 \\
 + [y]_{\text{补}} \quad 01001 \\
 \hline
 [x + y]_{\text{补}} \quad 10100
 \end{array}$$

两个正数相加的结果成为负数?

$x = -1101$, $y = -1011$, 求 $x + y$?

解: $[x]_{\text{补}} = 10011$, $[y]_{\text{补}} = 10101$

$$\begin{array}{r}
 [x]_{\text{补}} \quad 10011 \\
 + [y]_{\text{补}} \quad 10101 \\
 \hline
 [x + y]_{\text{补}} \quad 101000
 \end{array}$$

两个负数相加的结果成为正数?



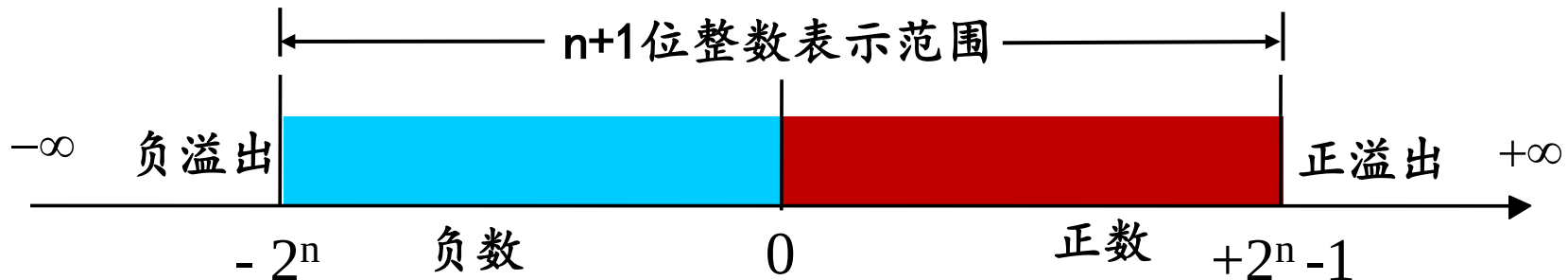


溢出概念

- 无论是定点小数还是定点整数，在运算过程中会出现运算结果超出机器表示范围的现象，称之为“溢出”

有符号整数加法运算：

- 正溢出：两个正数相加结果为负数（超出机器所能表示的范围）
- 负溢出：两个负数相加结果为正数（超出机器所能表示的范围）





溢出检测方法1：双符号位法（1）

■ 变形补码的定义

$$\text{小数: } [x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -2 \end{cases} \pmod{4}$$

$$\text{整数: } [x]_{\text{补}} = 2^{n+2} + x \pmod{2^{n+2}}$$

◆ 下式也同样成立

$$[x]_{\text{补}} + [y]_{\text{补}} = [x+y]_{\text{补}}$$

◆ 注意事项

- 两个符号位均参加运算
- 最高符号位上产生的进位要丢掉

00表示正，11表示负





溢出检测方法1：双符号位法（2）

■ 溢出检测规则

- ◆ 两数相加，运算结果的符号位为“01”或“10”时，表示发生溢出
- ◆ 最高符号位永远表示结果的正确符号

■ 溢出的逻辑表达式为

$$V = S_{f1} \oplus S_{f2}$$





举例

$$x = +1100$$

$$y = +1000, \text{ 求 } x + y$$

$$\text{解: } [x]_{\text{补}} = 001100$$

$$[y]_{\text{补}} = 001000$$

$$\begin{array}{r} [x]_{\text{补}} \quad 001100 \\ + [y]_{\text{补}} \quad 001000 \\ \hline [x + y]_{\text{补}} \quad 010100 \end{array}$$

两个符号位为“01”，表示已经溢出

最高有效位产生进位而
第1符号位无进位

$$x = -1100$$

$$y = -1000, \text{ 求 } x + y$$

$$\text{解: } [x]_{\text{补}} = 110100$$

$$[y]_{\text{补}} = 111000$$

$$\begin{array}{r} [x]_{\text{补}} \quad 110100 \\ + [y]_{\text{补}} \quad 111000 \\ \hline [x + y]_{\text{补}} \quad 1101100 \end{array}$$

两个符号位为“10”，表示已经溢出

最高有效位无进位而
第1符号位有进位





溢出检测方法2：单符号位法

- 溢出的逻辑表达式为

$$V = C_f \oplus C_0$$

C_f 为符号位产生的进位， C_0 为最高有效位产生的进位

现代处理器的判断方法





举例

■ 2个有符号数相加

00001001	00000010
+ 01111100	+ 11111111
10000101	10000001
V=1	V=0

✗

✓





动手做 (1)

- 某字长为 8 位的计算机中，已知整型变量 x 、 y 的机器数分别为 $[x]_{\text{补}} = 1111\ 0100$ ， $[y]_{\text{补}} = 1011\ 0000$ 。若整型变量 $z = 2 * x + y / 2$ ，则 z 的机器数为
- A. 1100 0000 B. 0100 0000
C. 1001 1000 D. 溢出





动手做 (2)

若 $x=103$, $y=-25$, 则下列表达式采用8位定点补码运算实现时, 会发生溢出的是

- A $x+y$
- B $-x+y$
- C $x-y$
- D $-x-y$





动手做 (3)

一个C语言程序在一台32位机器上运行。程序中定义了三个变量x、y、z，其中x和z是int型，y为short型。当x=127，y=-9时，执行赋值语句z=x+y后，x、y、z的值分别是

- A. x=0000007FH, y=FFF9H, z=00000076H
- B. x=0000007FH, y=FFF9H, z=FFFF0076H
- C. x=0000007FH, y=FFF7H, z=FFFF0076H
- D. x=0000007FH, y=FFF7H, z=00000076H

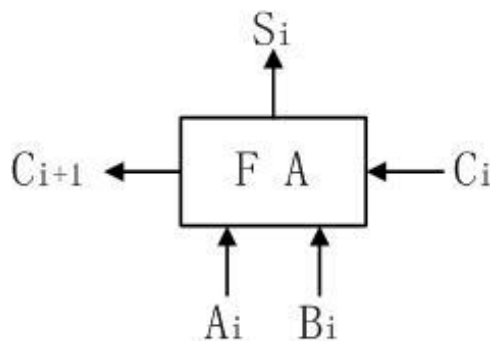




1位全加器FA (1)

$$\begin{aligned}
 S_i &= \overline{A_i} \overline{B_i} C_i + \overline{A_i} B_i \overline{C_i} \\
 &\quad + A_i \overline{B_i} \overline{C_i} + A_i B_i C_i \\
 &= A_i \oplus B_i \oplus C_i
 \end{aligned}$$

$$\begin{aligned}
 C_{i+1} &= \overline{A_i} \overline{B_i} C_i + A_i \overline{B_i} C_i \\
 &\quad + A_i \overline{B_i} \overline{C_i} + A_i B_i C_i \\
 &= A_i B_i + B_i C_i + C_i A_i \\
 &= A_i B_i + (A_i \oplus B_i) C_i
 \end{aligned}$$



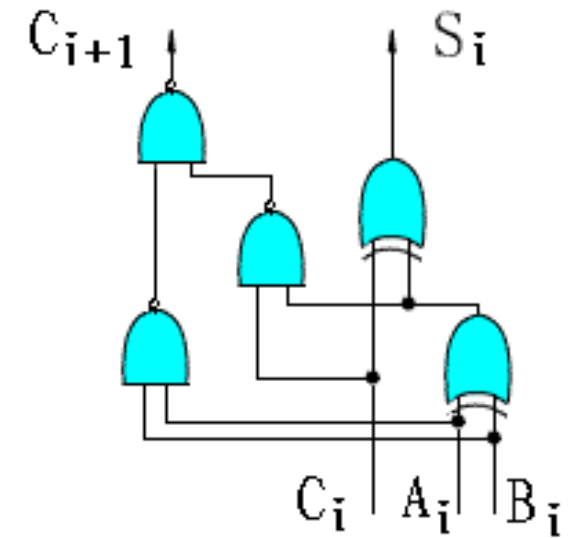
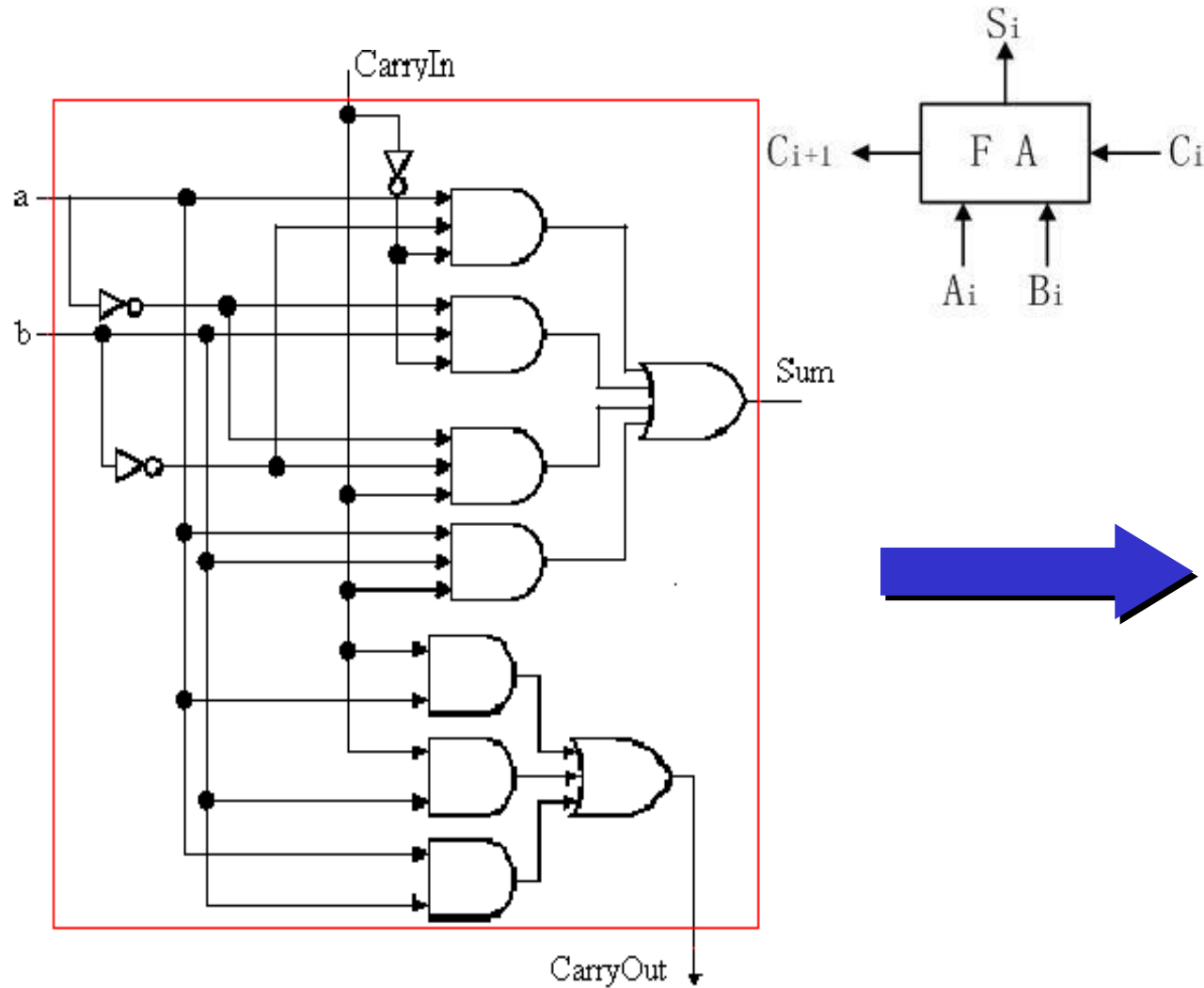
一位全加器真值表

输入			输出	
A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



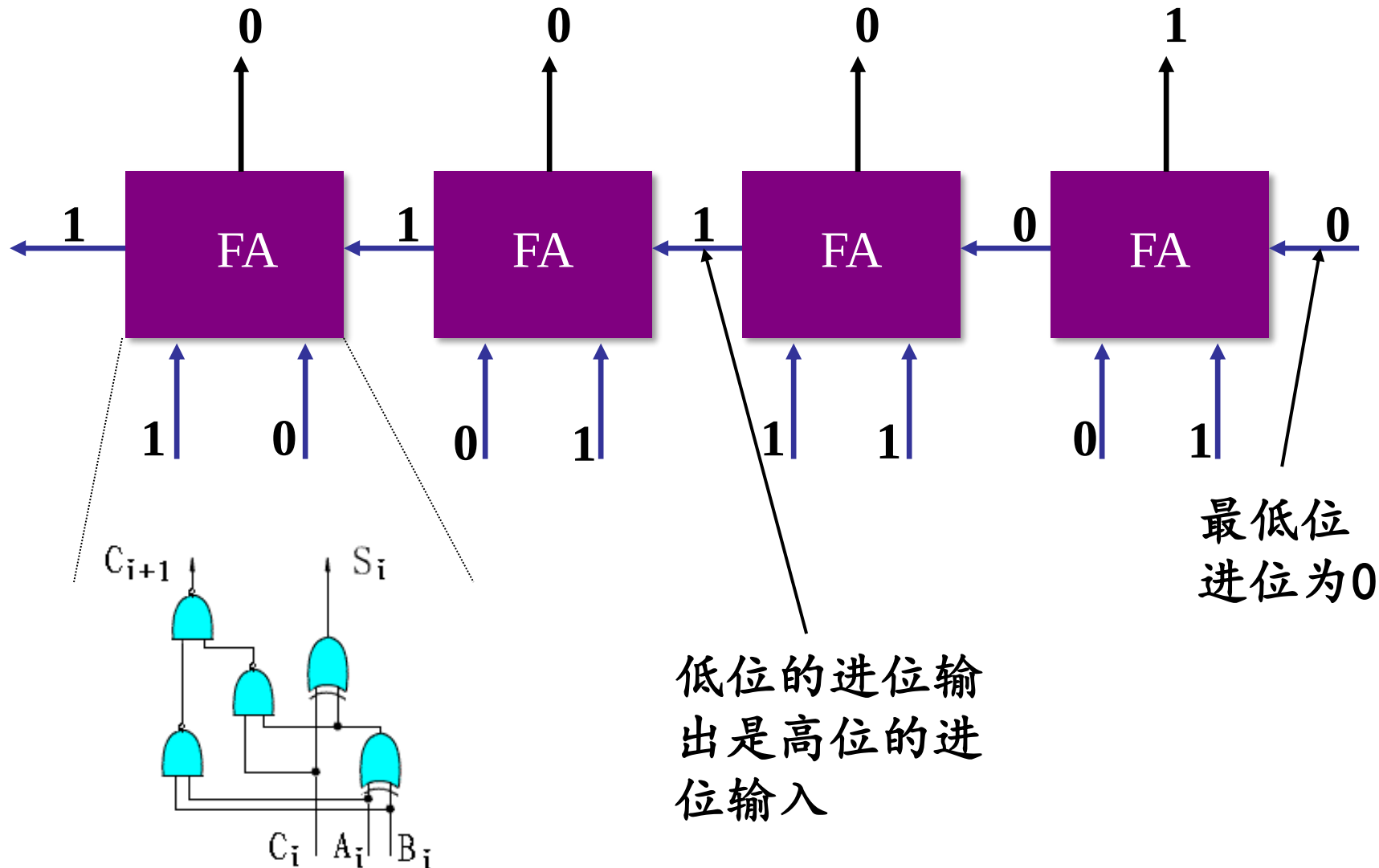


1位全加器FA (2)

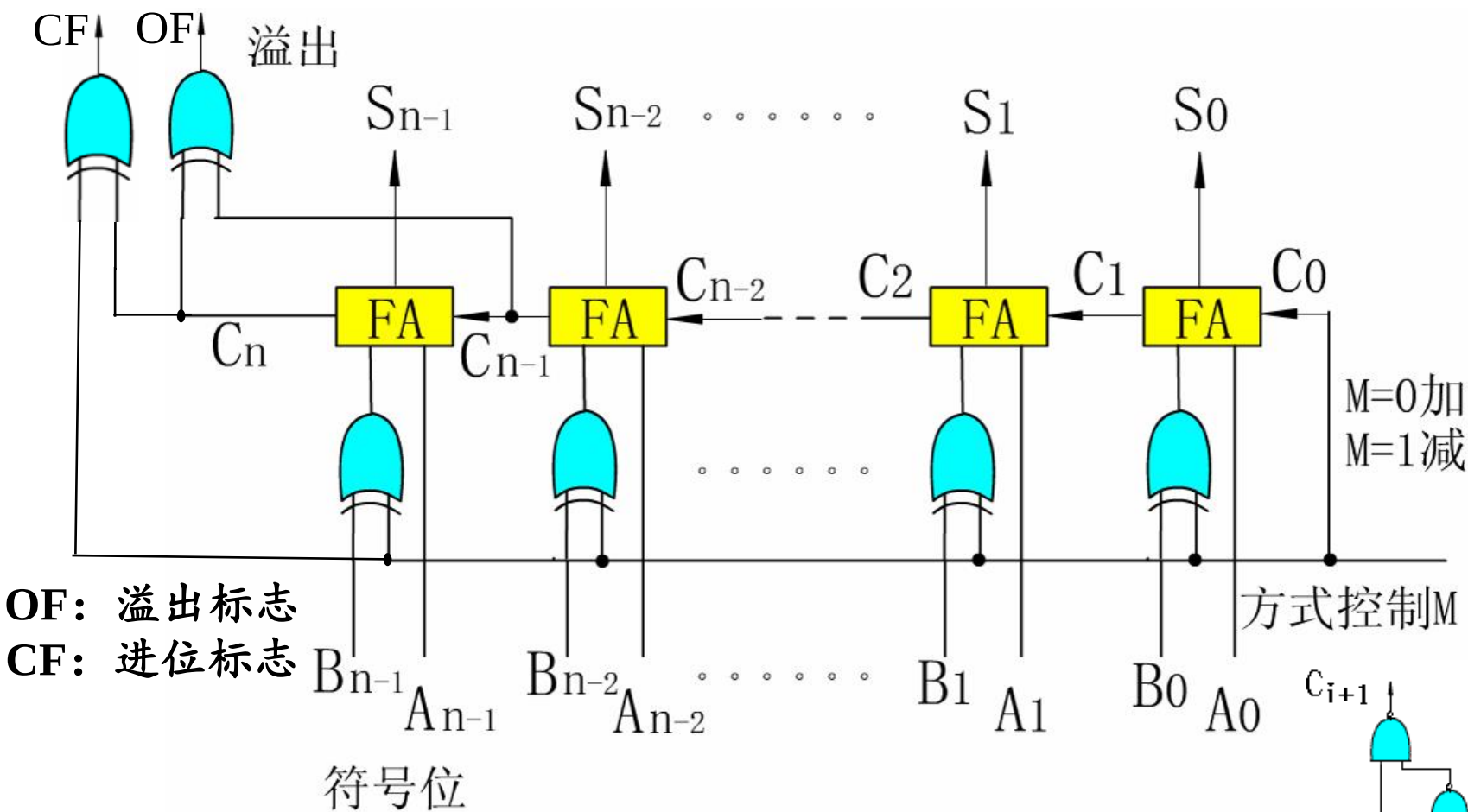




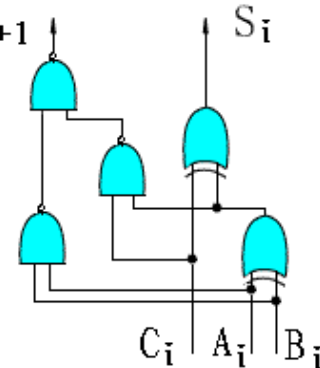
4位加法器设计



n位行波进位的补码加法器



$$[A]_{\text{补}} - [B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$$





n位行波进位加法器的时间延迟

- 考虑溢出检测，n位的行波进位加法器的时间延迟

n = 2时

$$\begin{aligned} t_a &= 3T + 3T + 2T + 2T + 3T \\ &= 6T + 2 \cdot 2T + 3T \end{aligned}$$

n位时： $t_a = 3T + 3T + n \cdot 2T + 3T = (2n + 9)T$

其中T为单级逻辑电路的单位门延迟，每级异或门延迟3T

- 不考虑溢出检测时，n位的行波进位加法器的时间延迟

$$t_a = 3T + 3T + 2T + 3T \quad (n=2时)$$

$$t_a = 3T + 3T + (n-1)2T + 3T = 2(n-1)T + 9T \quad (n位时)$$





改进 分析进位链逻辑

- 加法器进位链的基本逻辑关系

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

- 令 $G_i = A_i B_i$ 为进位产生函数（本地进位）

$P_i = A_i \oplus B_i$ 为进位传递函数（进位条件）

即： $P_i C_i$ 为传送进位或条件进位

于是， $C_{i+1} = G_i + P_i C_i$

- 只有当 $A_i = B_i = 1$ 时，本位才向高位进位
- 当 $A_i \neq B_i$ 时，低一位的进位将向更高位传送
- 传送进位和本地进位不可能同时为1





进位产生和传递函数

$$C_{i+1} = G_i + P_i C_i = G_i + P_i (G_{i-1} + P_{i-1} C_{i-1})$$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

.....

- 依此公式递推，所有进位均可从最低进位 C_0 直接求得而无需等待低一位进位
- 对任意 i ，均可得到只含 G_k 、 P_k ($k=0\sim i$) 以及 C_0 的 C_{i+1} 的表达式
- 先行进位逻辑的总时间延迟为 $5T = (3T+2T)$
每级与门或或门的延迟为 T ，每级异或门延迟 $3T$





4位先行进位部件CLA

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

$$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0 + P3P2P1P0C0$$

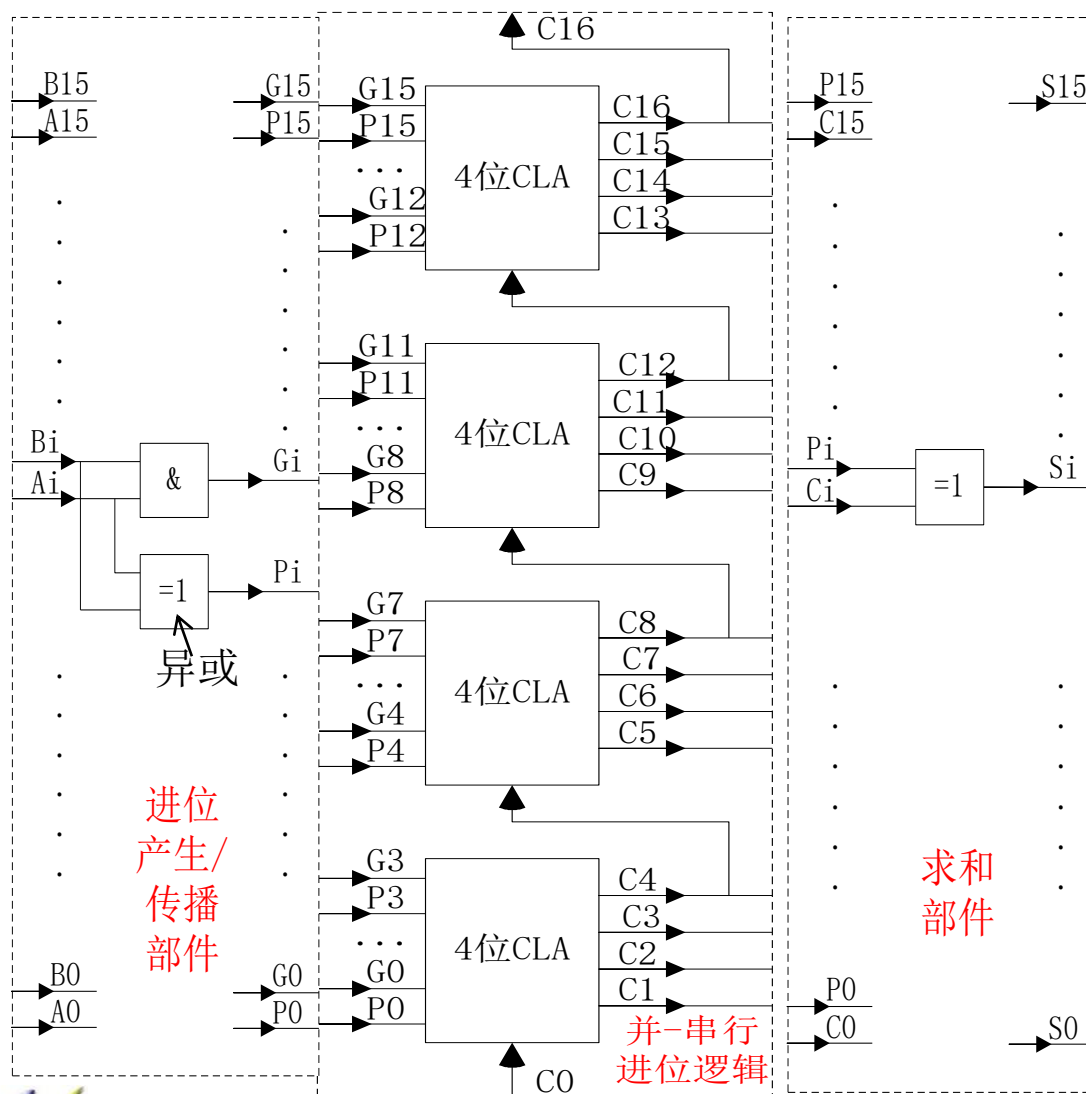
延迟时间：2T（不含求 P_i 的时间）

CLA: Carry Look Ahead





16位单级分组先行进位加法器



分成4个进位组，组内并行进位，组间串行进位。延迟时间：

• 先行进位部件：

$$2T \times 4 = 8T$$

• 进位产生/传播部件：3T

• 求和部件：3T

• 加法器的总延迟时间：

$$t = 3T + 4 \times 2T + 3T = 14T$$

比较：

• 串行进位16位加法器的总延迟时间：

$$t = 9T + 2(n-1)T$$

$$= 9T + 2 \times 15T = 39T$$





2.5 定点运算器的组成





运算器的基本结构

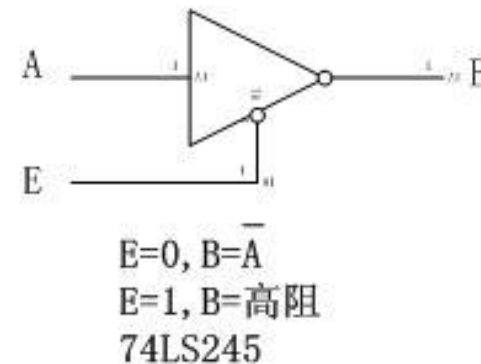
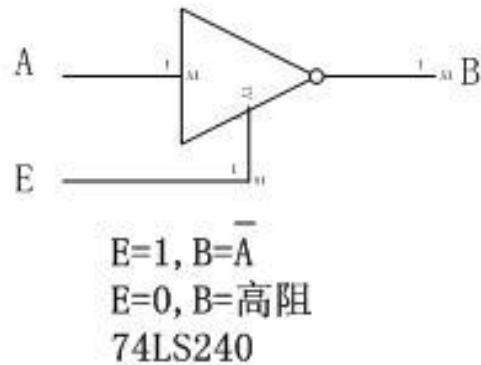
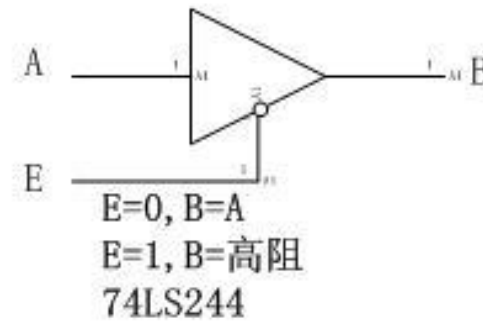
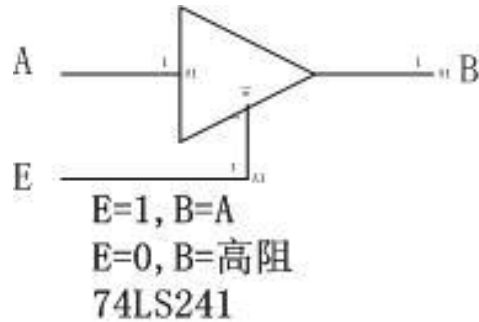
典型运算器包括：

- 算术逻辑运算单元ALU
- 阵列乘法器
- 寄存器组
- 多路选择器
- 三态门
- 数据总线





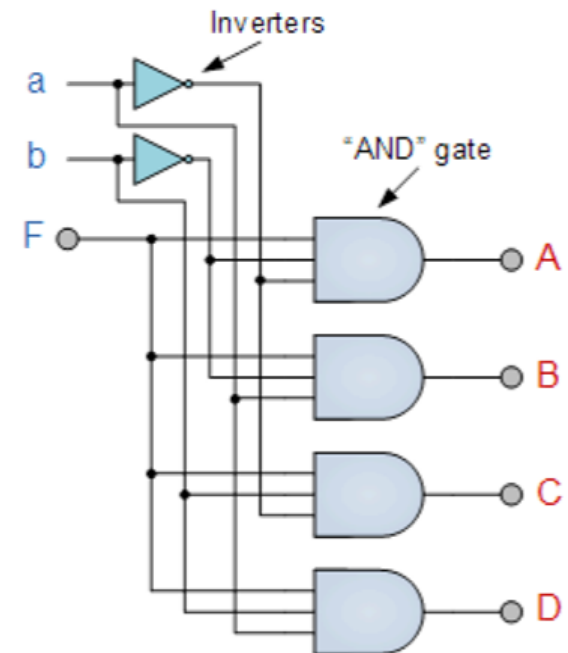
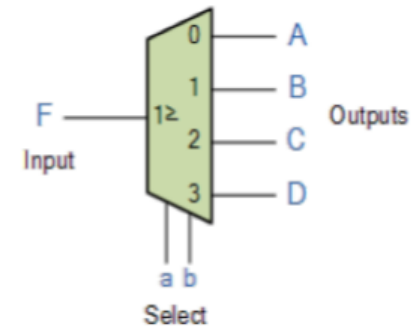
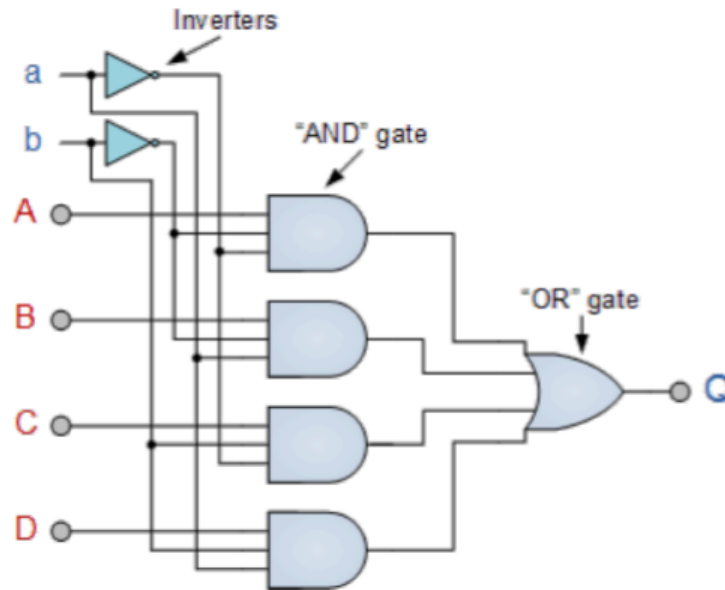
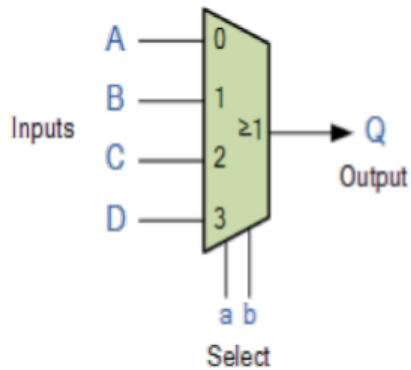
三态门





多路复用器/多路分配器

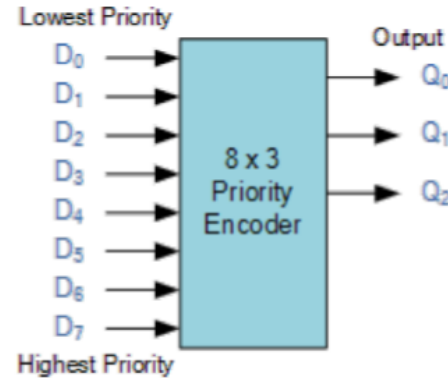
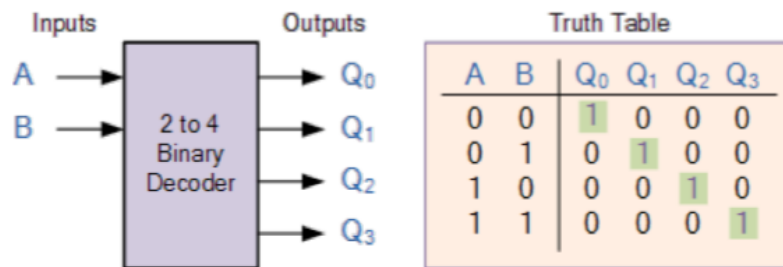
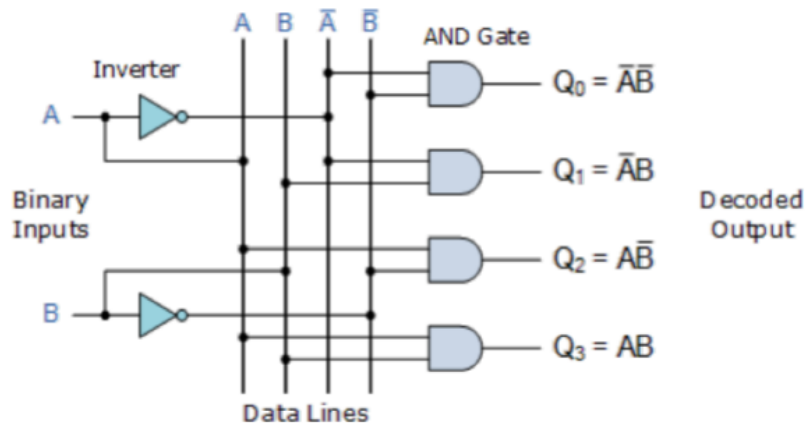
Multiplexor/Demultiplexor





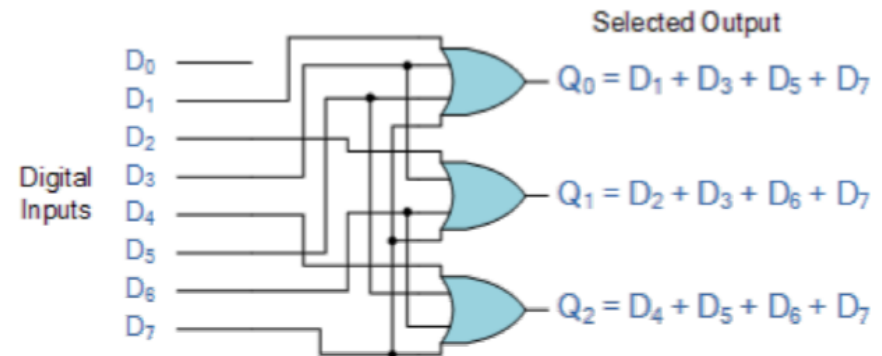
译码器/编码器

Decoder/Encoder



Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	1	x	x	x	x	1	0	0
0	0	1	x	x	x	x	x	1	0	1
0	1	x	x	x	x	x	x	1	1	0
1	x	x	x	x	x	x	x	1	1	1

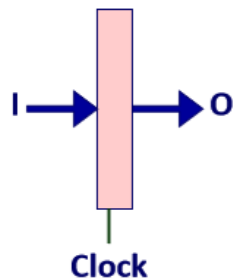
X = don't care



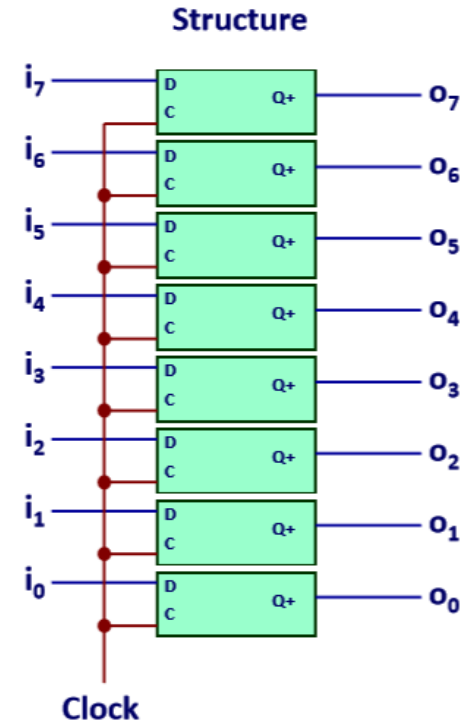


寄存器

Registers



- Stores data bits
- For most of time acts as barrier between input and output
- As clock rises, loads input

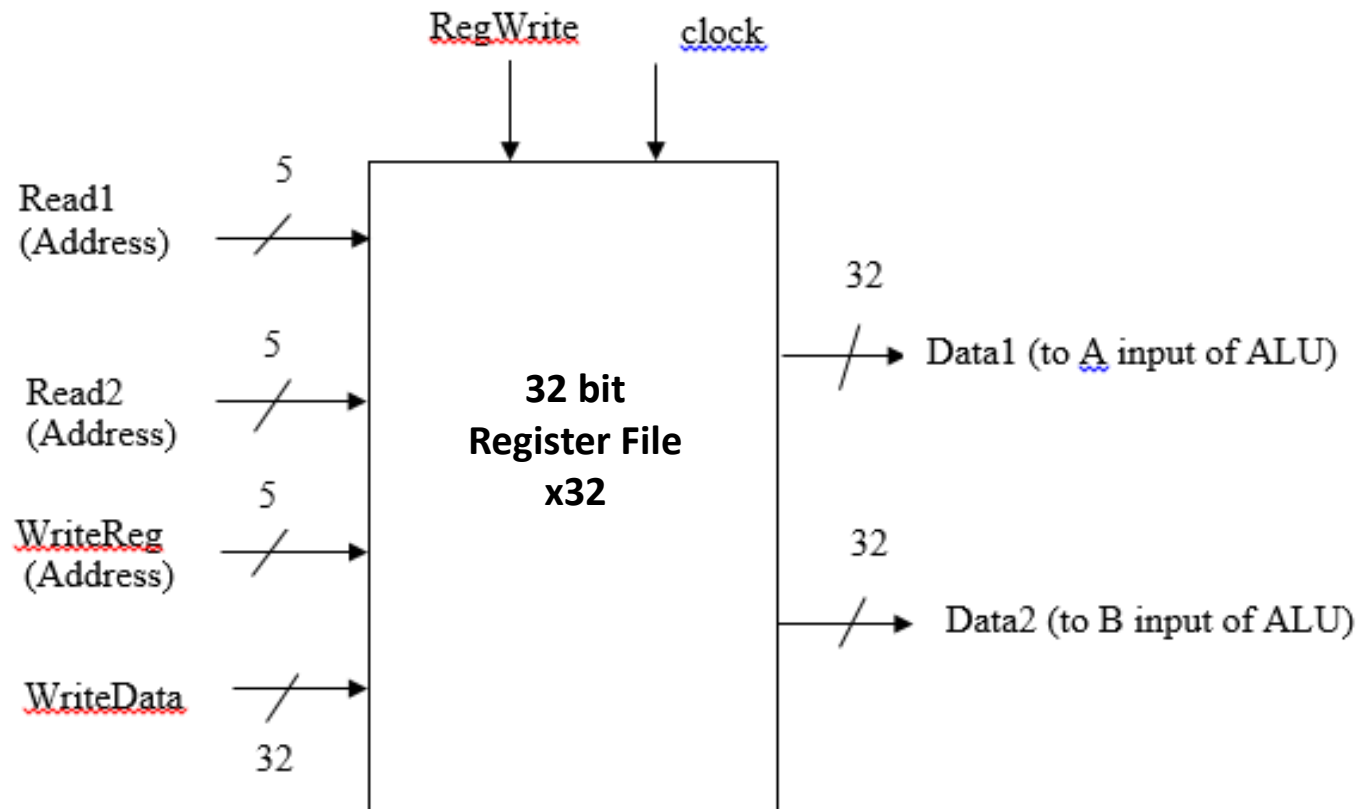


- Stores word of data
 - Different from *program registers* seen in assembly code
- Collection of edge-triggered latches
- Loads input on rising edge of clock

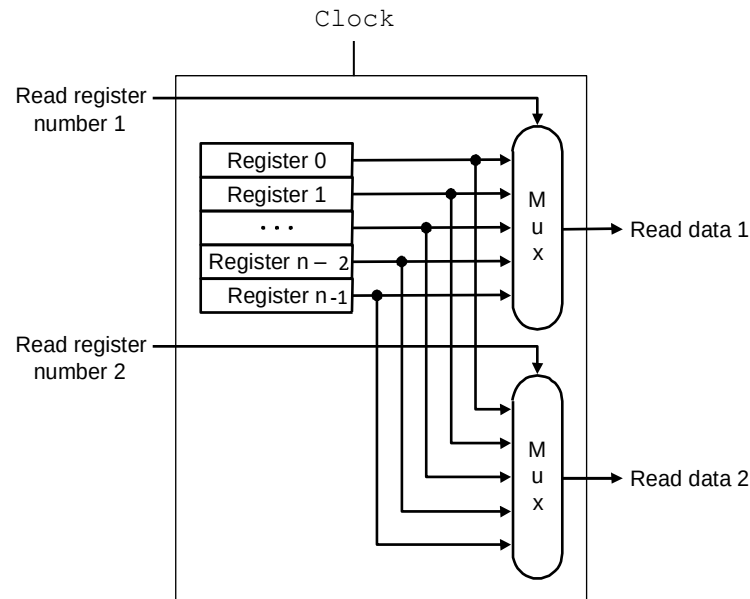




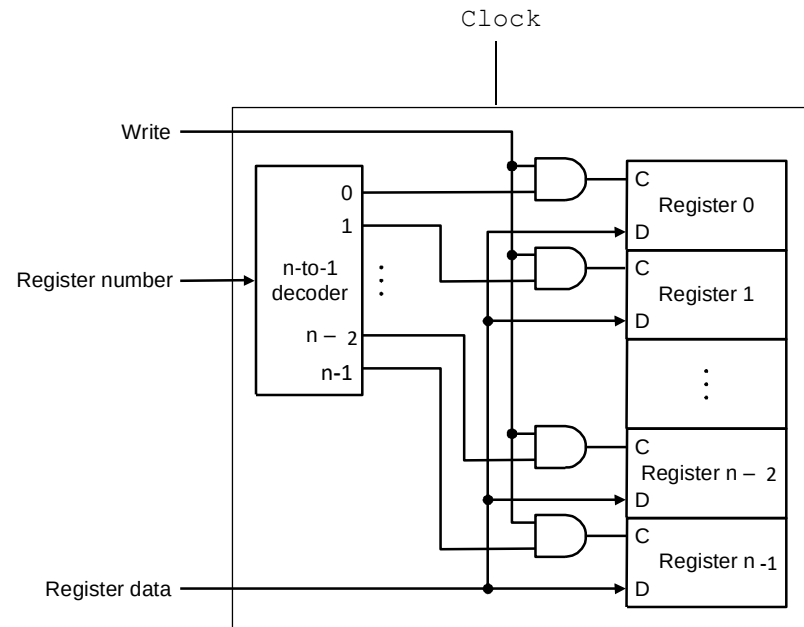
寄存器组



寄存器组内部实现



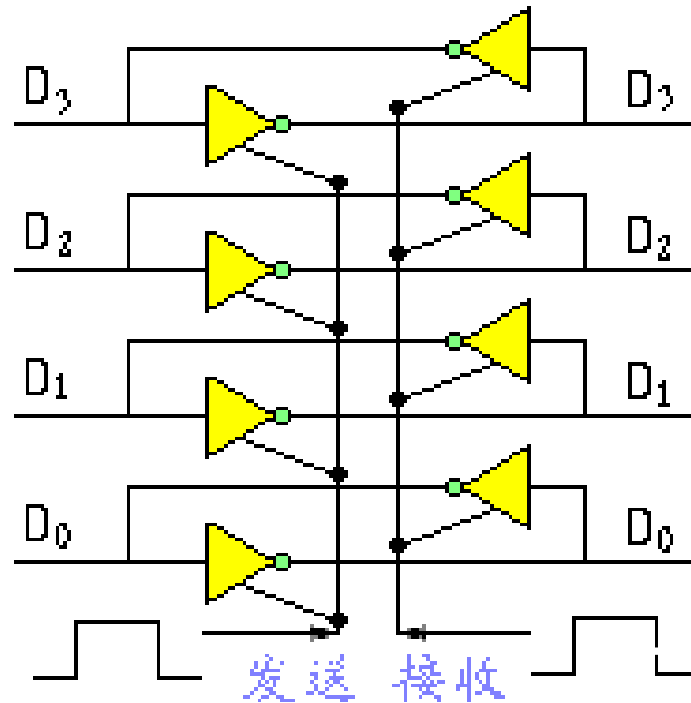
Read ports are implemented with a pair of multiplexors – 5 bit multiplexors for 32 registers



Write port is implemented using a decoder – 5-to-32 decoder for 32 registers. Clock is relevant to write as register state may change only at clock edge



总线驱动器



带有缓冲器的双向数据总线



内部总线

- 机器内部各部件数据传送频繁，可以把寄存器间的数据传送通路加以归并，组成总线结构

- 总线分类

- ◆ 所处位置

- 内部总线（CPU内）
 - 外部总线（系统总线）

- ◆ 逻辑结构

- 单向传送总线
 - 双向传送总线





逻辑运算

- 逻辑非: $x = x_0x_1..x_n$, 则 $\bar{x} = \bar{x}_0\bar{x}_1..\bar{x}_n$
对某数进行逻辑非运算, 就是按位求它的反
- 逻辑或: $1 + x = 1$, $0 + x = x$
将一个数的某些指定比特置1, 其他比特保持不变
例如: $a + 01000010 \leftrightarrow$ 将a的第1和第6位置1
- 逻辑与: $1 * x = x$, $0 * x = 0$
将一个数的某些指定比特清0, 其他比特保持不变
例如: $a * 10111101 \leftrightarrow$ 将a的第1和第6位清0
- 逻辑异或: $1 \oplus x = \bar{x}$, $0 \oplus x = x$
将一个数的某些指定比特取反, 其他比特保持不变
例如: $a \oplus 01000010 \leftrightarrow$ 将a的第1和第6位取反





一位全加器FA

■ 逻辑表达式

$$F_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + C_i A_i$$

只能完成加、减运算

■ 一种解决方法

为了实现多种算术逻辑运算，可先将 A_i 和 B_i 输入到一个函数发生器得到输出 X_i 和 Y_i ，再将 X_i 和 Y_i 作为一位全加器的输入

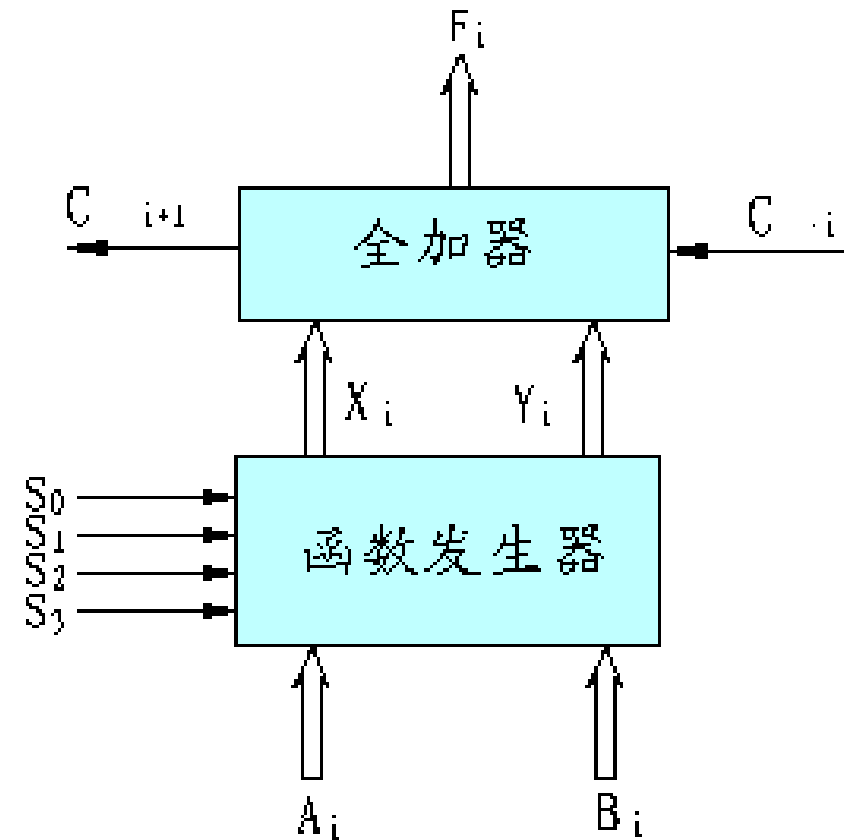




算术/逻辑运算单元ALU (1)

基本思想

- 先将原全加器的输入 A_i 、 B_i 送至函数发生器进行处理
- S_0 、 S_1 、 S_2 、 S_3 为函数发生器的控制信号
- 函数发生器的输出 X_i 、 Y_i 至全加器进行全加
- 由 S_0 、 S_1 、 S_2 、 S_3 来控制ALU所完成的运算



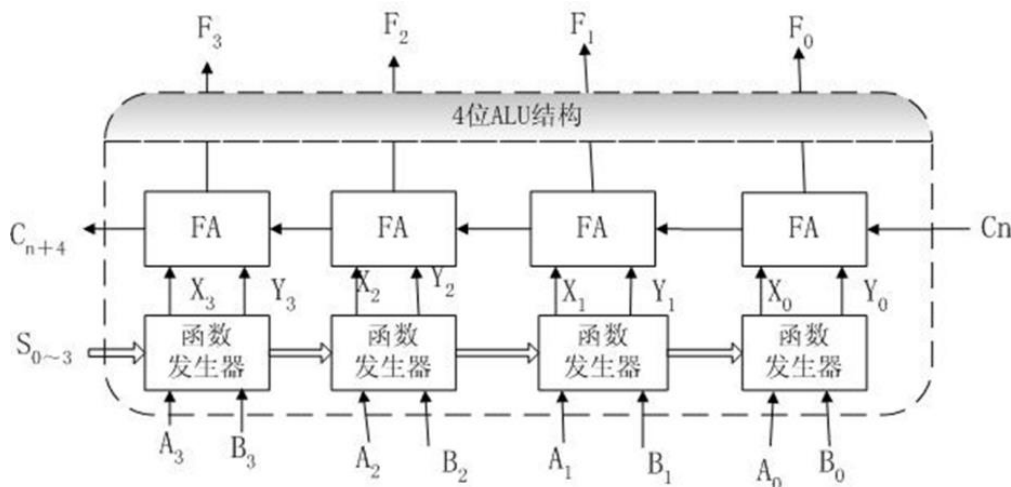


算术/逻辑运算单元ALU (2)

■ $F_i = X_i \oplus Y_i \oplus C_{n+i}$

$$C_{n+i+1} = X_i Y_i + Y_i C_{n+i} + X_i C_{n+i}$$

- ◆ 用 $n+i$ 代替原来一位全加器 C_i 中的 i ， i 表示在一片ALU芯片上二进制位数。对于一片4位ALU， $i=0、1、2、3$ ； n 表示用多片ALU组成更多位数运算器时每片电路的进位输入。如果用4片4位ALU组成一个16位运算器， $n=0、4、8、12$





函数发生器逻辑表达式

■ X_i 、 Y_i 与控制参数和输入量的关系

$S_0 S_1$	Y_i	$S_2 S_3$	X_i
0 0	$\bar{A}_i$	0 0	1
0 1	$\bar{A}_i B_i$	0 1	$\bar{A}_i + \bar{B}_i$
1 0	$\bar{A}_i \bar{B}_i$	1 0	$\bar{A}_i + B_i$
1 1	0	1 1	$\bar{A}_i$

$$X_i = \bar{S}_2 \bar{S}_3 + \bar{S}_2 S_3 (\bar{A}_i + \bar{B}_i) + S_2 \bar{S}_3 (\bar{A}_i + B_i) + S_2 S_3 \bar{A}_i$$

$$= S_3 A_i B_i + S_2 A_i B_i$$

$$Y_i = \bar{S}_0 \bar{S}_1 \bar{A}_i + \bar{S}_0 S_1 \bar{A}_i B_i + S_0 \bar{S}_1 \bar{A}_i \bar{B}_i$$

$$= A_i + S_0 B_i + \bar{S}_i B_i$$

$$X_i Y_i = Y_i \Rightarrow C_{n+i+1} = X_i Y_i + Y_i C_{n+i} + X_i C_{n+i} = Y_i + X_i C_{n+i}$$





ALU的第i位逻辑表达式

$$\begin{cases} X_i = \overline{S_3 A_i B_i + S_2 A_i \overline{B_i}} \\ Y_i = \overline{A_i + S_0 B_i + S_1 \overline{B_i}} \\ F_i = X_i \oplus Y_i \oplus C_{n+i} \\ C_{n+i+1} = Y_i + X_i C_{n+i} \end{cases} \quad \begin{matrix} i=0,1,2,3 \\ n=0,4,8,12 \end{matrix}$$

例如: $S_3 S_2 S_0 S_1 = 0000$

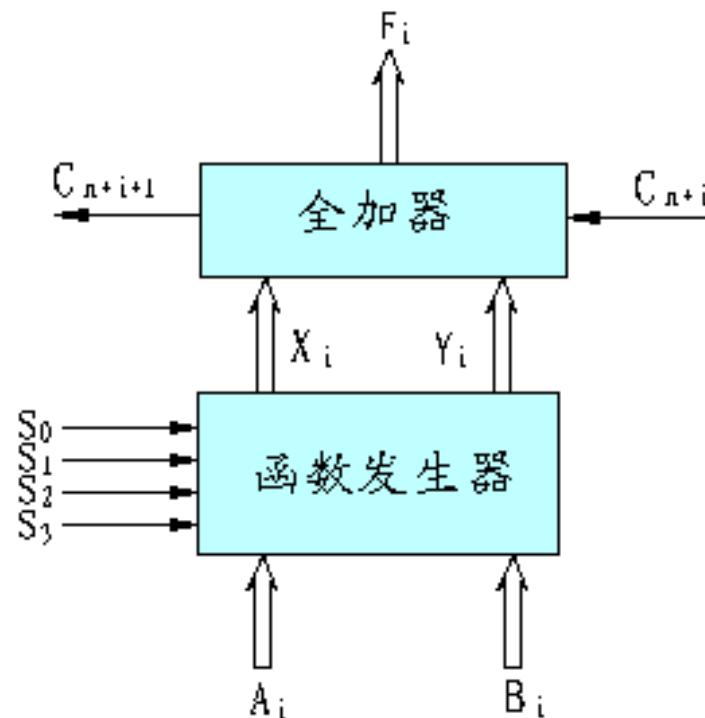
代入:

$$X_i = \overline{S_3 A_i B_i + S_2 A_i \overline{B_i}} = \overline{0 + 0} = 1$$

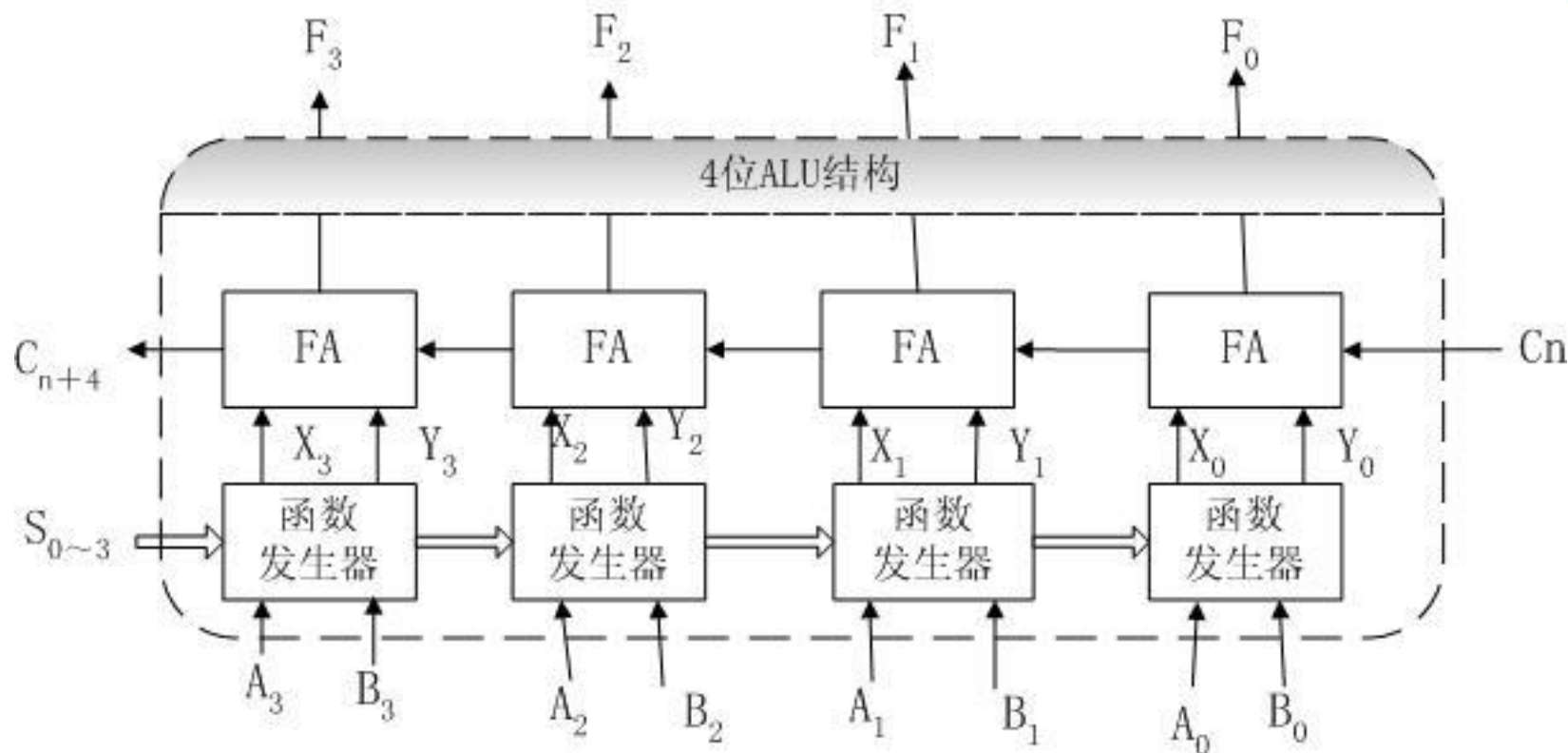
$$Y_i = \overline{A_i + S_0 B_i + S_1 \overline{B_i}} = \overline{A_i}$$

$$F_i = Y_i \oplus X_i \oplus C_{n+1} = \overline{A_i} \oplus 1 \oplus C_{n+1} = A_i \text{ (设 } C_{n+1}=0\text{)}$$

所以, 在 $S_3 \sim S_0 = 0000$ 的时候, 输出结果为 A_i



构造一种简单的4位ALU结构



■ 片内是串行进位还是并行进位？

串行进位，速度慢，可以改进

$$C_{n+1} = Y_0 + X_0 C_n$$

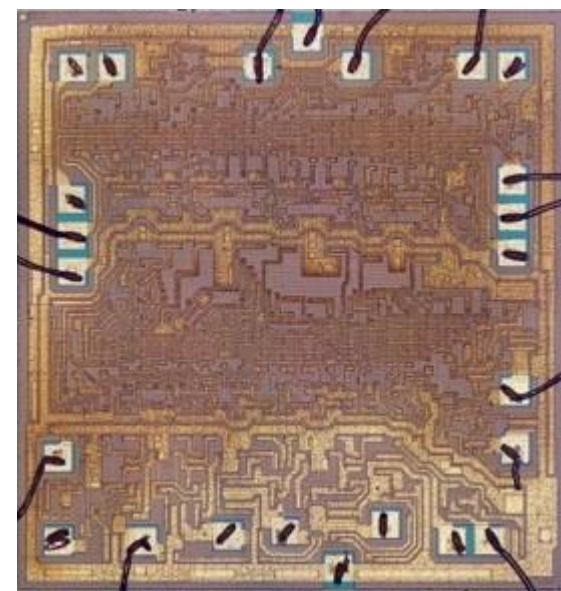
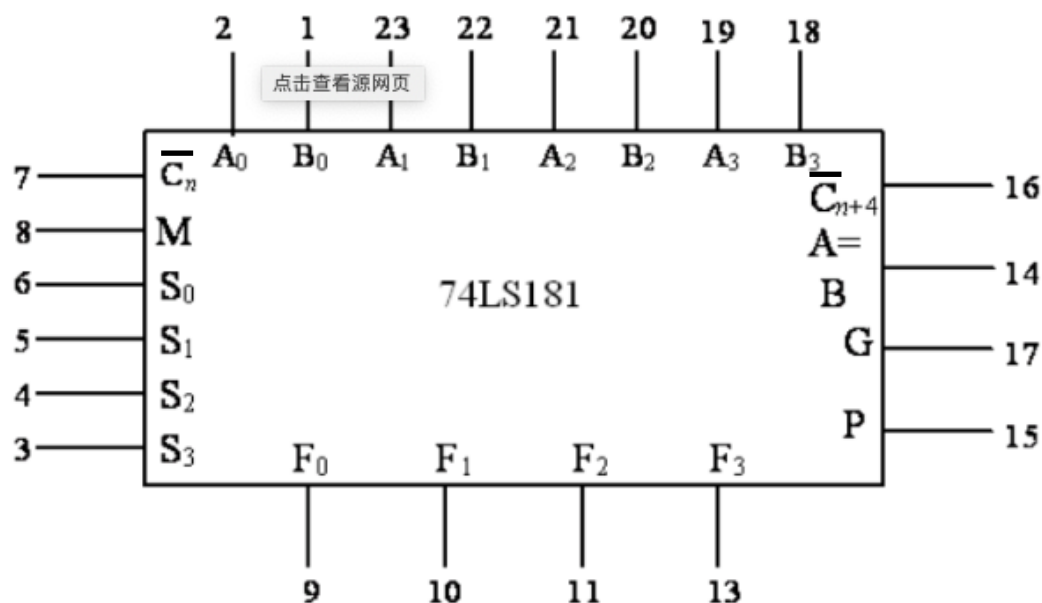
$$C_{n+2} = Y_1 + X_1 C_{n+1}$$

$$C_{n+3} = Y_2 + X_2 C_{n+2}$$

$$C_{n+4} = Y_3 + X_3 C_{n+3}$$

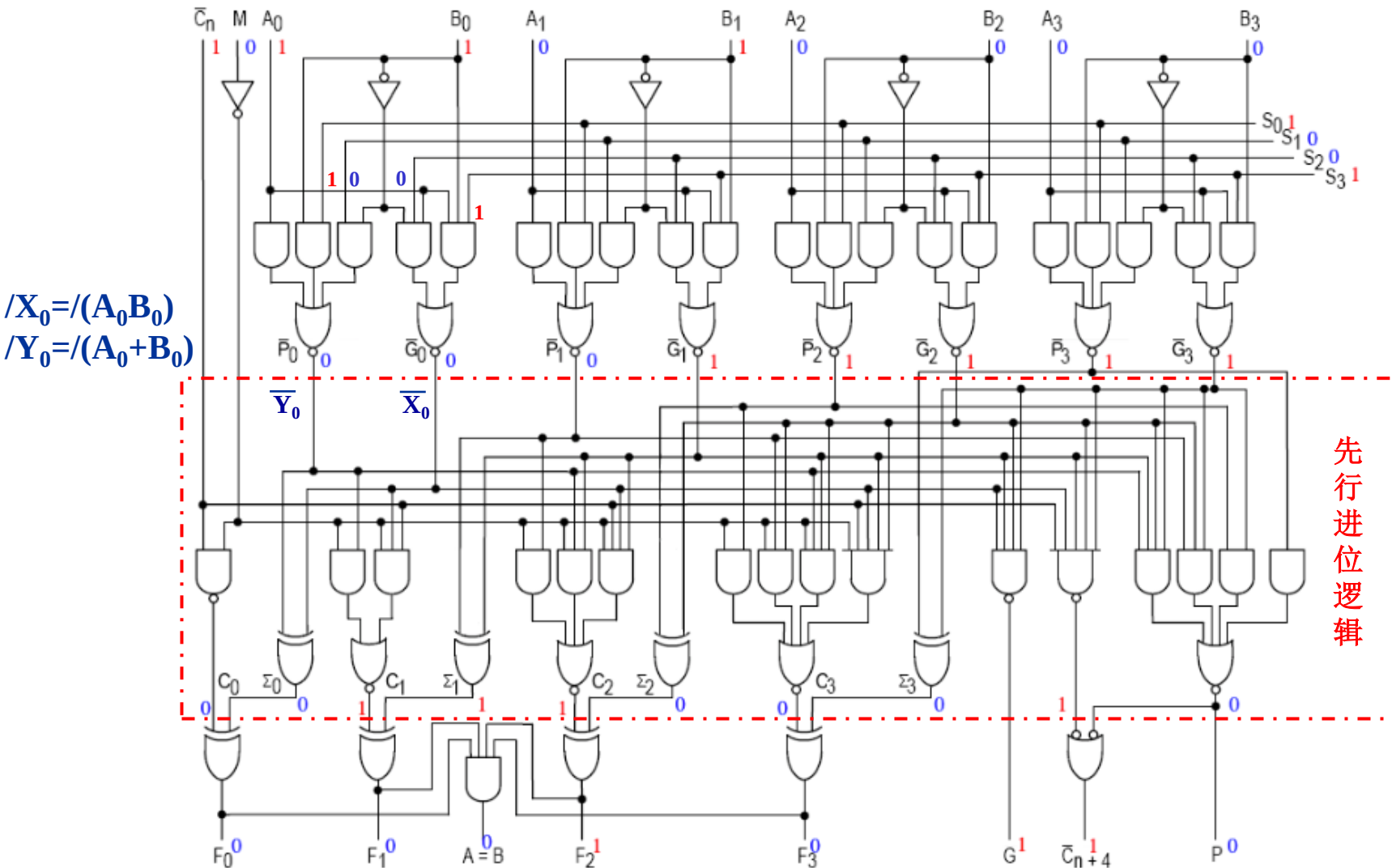
4位算术/逻辑运算单元实例

■ 74181





74181ALU逻辑图



例：当 $S_{0-3}=1001$ 、 $M=0$ 、 $/C_n=1$ 时， $F_0 = /X_0 \oplus /Y_0 \oplus C_0 = /X_0 \oplus /Y_0 = /(A_0 B_0) \oplus /(A_0 + B_0) = A_0 \oplus B_0$



采用先行进位

■ 每一位的进位公式：

$$C_{n+1} = Y_0 + X_0 C_n$$

$$C_{n+2} = Y_1 + X_1 C_{n+1} = Y_1 + Y_0 X_1 + X_0 X_1 C_n$$

$$C_{n+3} = Y_2 + X_2 C_{n+2} = Y_2 + Y_1 X_2 + Y_0 X_1 X_2 + X_0 X_1 X_2 C_n$$

$$\begin{aligned} C_{n+4} &= Y_3 + X_3 C_{n+3} = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3 + X_0 X_1 X_2 X_3 C_n \\ &= G + P C_n \end{aligned}$$

其中， $G = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3$ ， $P = X_0 X_1 X_2 X_3$

从上式可以看出：第0位的进位输入 C_n 可以直接传送到最高进位位上去，从而实现先行进位

■ 4位ALU，有3个进位输出

◆ C_{n+4} 进位、本地进位 G 、进位传送 P

◆ 增加 P 、 G 是为了在多片ALU之间实现先行进位





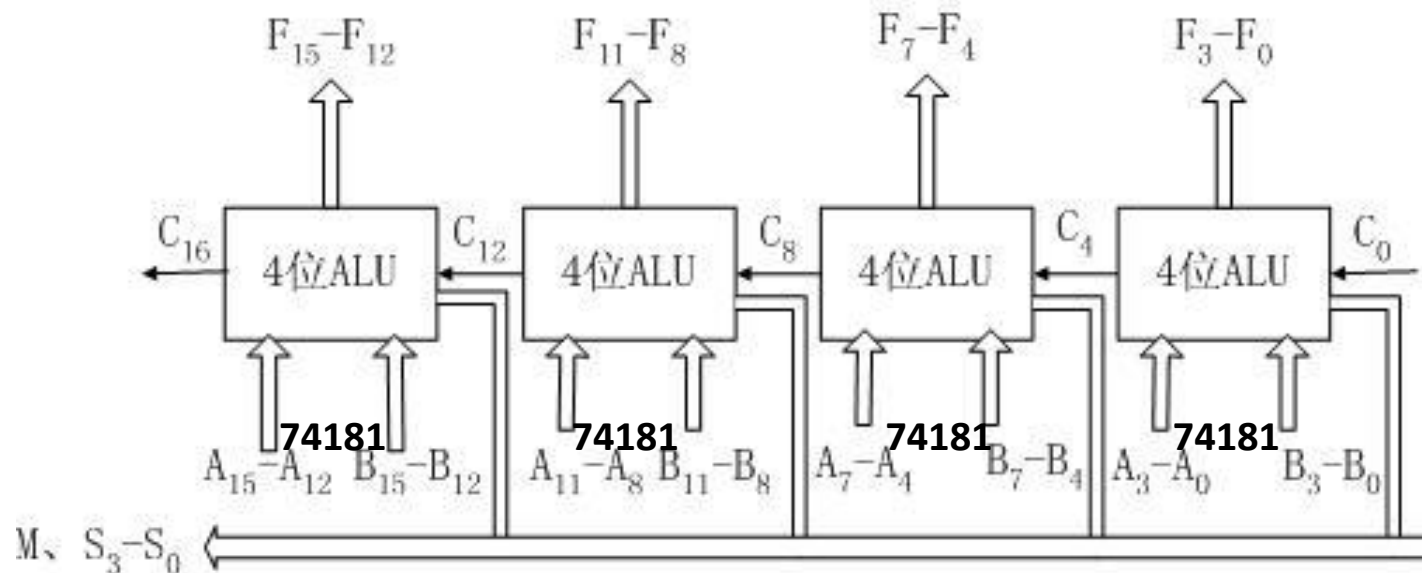
算术逻辑运算的实现

- **M=0**时，对进位信号没有影响，完成算术运算
- **M=1**时，进位位门被封锁，完成逻辑运算
- 说明：
 - ◆ **A=B**端可以判断两个数是否相等

方式	M=1 逻辑运算	M=0 算术运算	
S3 S2 S1 S0	逻辑运算	/CN=1 (无进位)	/CN=0 (有进位)
0 0 0 0	$F = /A$	$F = /A$	$F = A \text{ 加 } 1$
0 0 0 1	$F = /(A+B)$	$F = (A+B)$	$F = (A+B) \text{ 加 } 1$
0 0 1 0	$F = (/A)B$	$F = A+/B$	$F = (A+/B) \text{ 加 } 1$
0 0 1 1	$F = 0$	$F = \text{负 } 1 \text{ (补码形式)}$	$F = 0$
0 1 0 0	$F = /(AB)$	$F = A \text{ 加 } A(/B)$	$F = A \text{ 加 } A/B \text{ 加 } 1$
0 1 0 1	$F = /B$	$F = (A+B) \text{ 加 } A/B$	$F = (A+B) \text{ 加 } A/B \text{ 加 } 1$
0 1 1 0	$F = A \oplus B$	$F = A \text{ 减 } B \text{ 减 } 1$	$F = A \text{ 减 } B$
0 1 1 1	$F = A/B$	$F = A(/B) \text{ 减 } 1$	$F = A(/B)$
1 0 0 0	$F = /A+B$	$F = A \text{ 加 } AB$	$F = A \text{ 加 } AB \text{ 加 } 1$
<u>1 0 0 1</u>	$F = /(A \oplus B)$	<u>$F = A \text{ 加 } B$</u>	$F = A \text{ 加 } B \text{ 加 } 1$
1 0 1 0	$F = B$	$F = (A+/B) \text{ 加 } AB$	$F = (A+/B) \text{ 加 } AB \text{ 加 } 1$
1 0 1 1	$F = AB$	$F = AB \text{ 减 } 1$	$F = AB$
1 1 0 0	$F = 1$	$F = A \text{ 加 } A$	$F = A \text{ 加 } A \text{ 加 } 1$
1 1 0 1	$F = A+/B$	$F = (A+B) \text{ 加 } A$	$F = (A+B) \text{ 加 } A \text{ 加 } 1$
1 1 1 0	$F = A+B$	$F = (A+/B) \text{ 加 } A$	$F = (A+/B) \text{ 加 } A \text{ 加 } 1$
1 1 1 1	$F = A$	$F = A \text{ 减 } 1$	$F = A$



设计一个16位ALU



- 进位方式？
- 片内先行进位，片间串行进位

片间串行进位是否可以进一步改进？



改进 两级先行进位16位ALU

- 74181ALU已设置了P和G两个本组先行进位输出端
- 将4片74181的P、G输出端送入到74182先行进位部件（CLA），实现第二级的先行进位，即组与组之间的先行进位





74182CLA的进位逻辑关系

■ 可接4片ALU的先行进位逻辑

$$C_{n+x} = G_0 + P_0 C_n$$

$$C_{n+y} = G_1 + P_1 C_{n+x} = G_1 + G_0 P_1 + P_0 P_1 C_n$$

$$\begin{aligned} C_{n+z} &= G_2 + P_2 C_{n+y} \\ &= G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_n \end{aligned}$$

$$\begin{aligned} C_{n+4} &= G_3 + P_3 C_{n+z} \\ &= G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + P_0 P_1 P_2 P_3 C_n \\ &= G^* + P^* C_n \end{aligned}$$

其中， G_i 和 P_i 表示第*i*片ALU的本地进位G、进位传送P

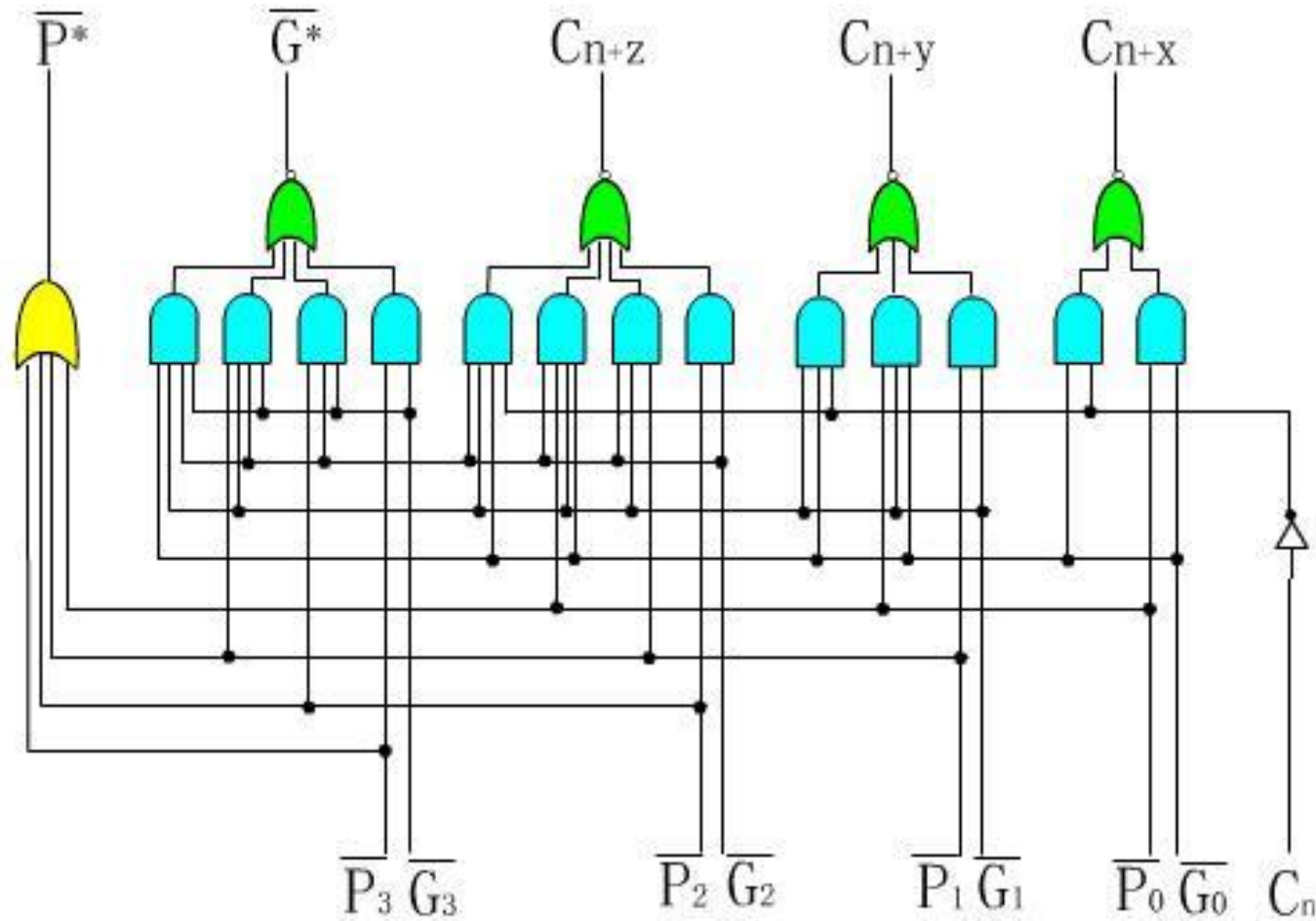
G^* 为成组先行进位发生输出
 P^* 为成组先行进位传送输出

用于更高一级先行进位





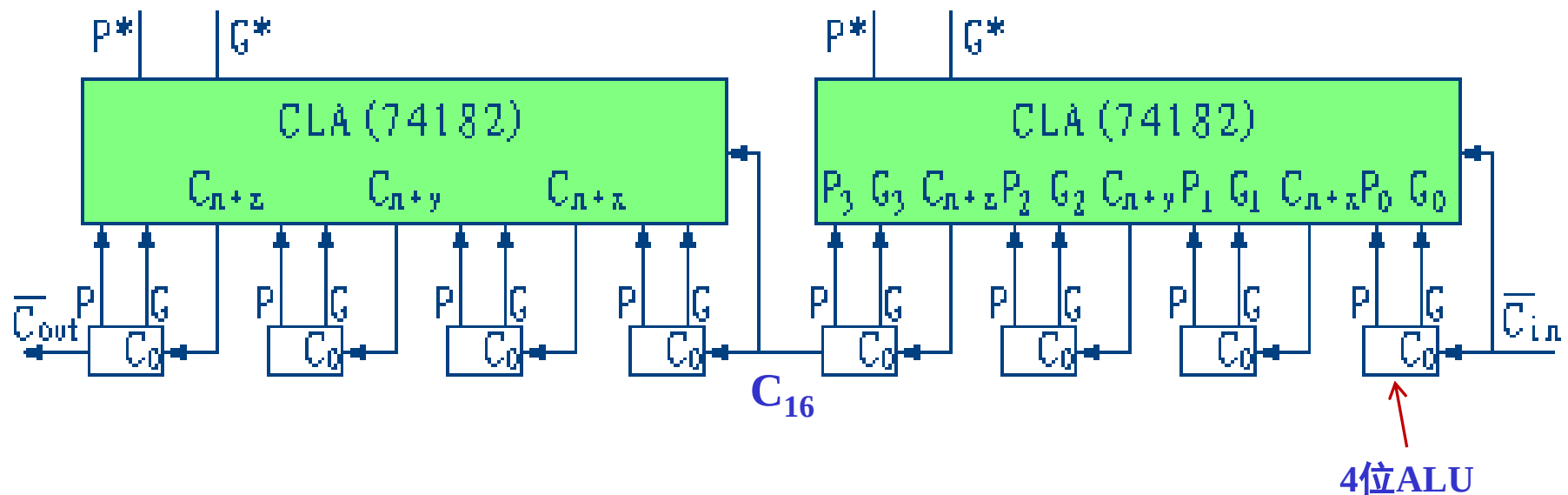
74182CLA的逻辑电路图





设计一个32位ALU

- 用两个16位全先行进位逻辑级联组成的32位ALU
即：构成的16位ALU组内为先行进位，组间为串行

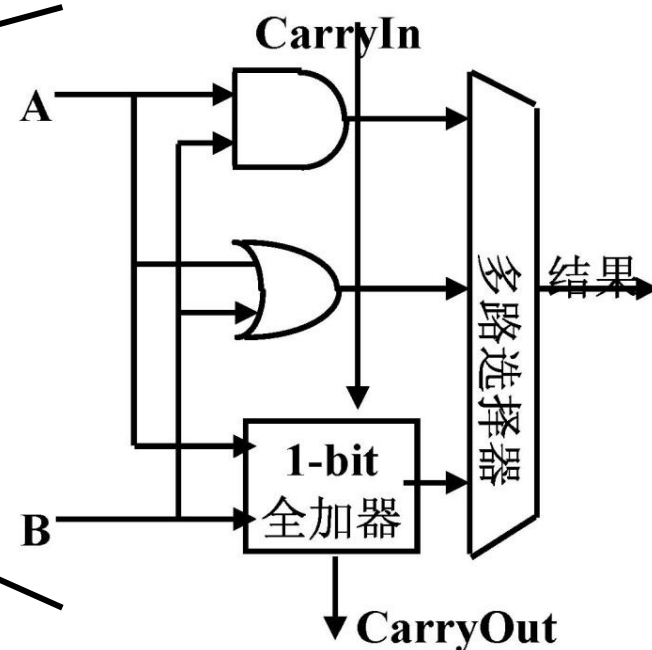
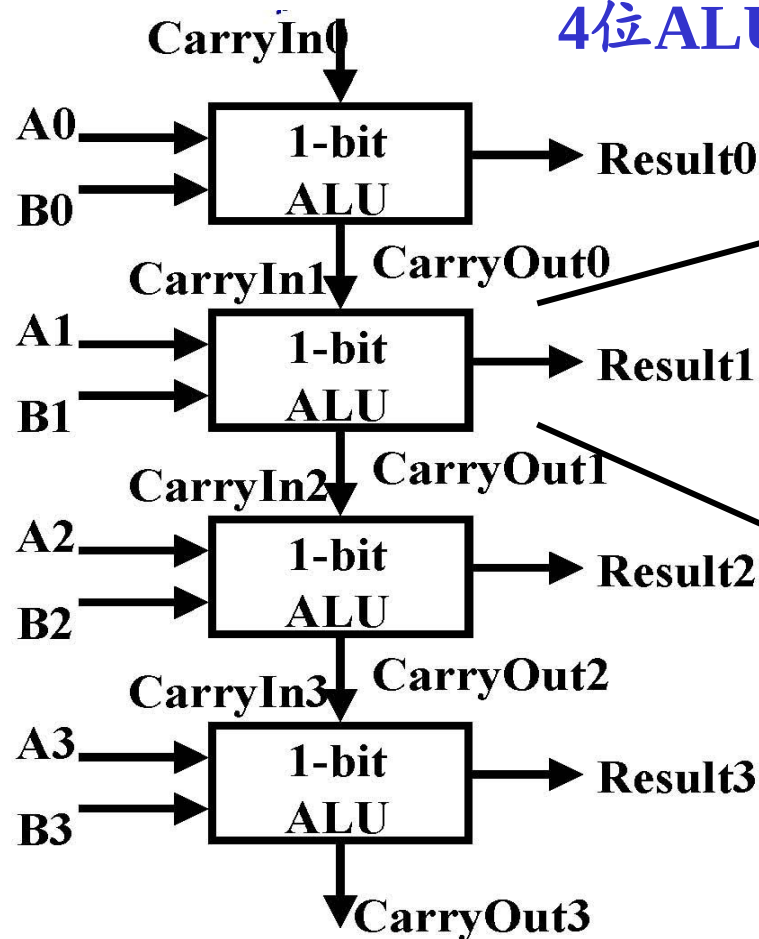




4位ALU的另一种方案

- 实现功能简单 $\Rightarrow$ 逻辑电路简单

4位ALU





BCD码加法实例

■ $x=36, y=47$, 求 $x+y$

解: $[x]_{\text{BCD}}=0011\ 0110, [y]_{\text{BCD}}=0100\ 0111$

$$\begin{array}{r}
 [x]_{\text{BCD}} \quad 0011\ 0110 \\
 +[y]_{\text{BCD}} \quad 0100\ 0111 \\
 \hline
 \quad \quad \quad 0111\ 1101
 \end{array}$$

$$\begin{array}{r}
 \text{校正} \quad \quad \quad +\ 0110 \\
 \hline
 [x+y]_{\text{BCD}} \quad 1000\ 0011
 \end{array}$$

所以, $x+y=83$

校正逻辑: BCD码完成十进制运算时, 当和数大于9时, 必须对和数进行加6修正





1位十进制（BCD）加法器设计

令 X_i 和 Y_i 为两个一位BCD码。

■ 设 S_i 为BCD码的和， C_{i+1} 为BCD码的进位输出， C_i 为进位输入

■ 令 S'_i 为修正之前的二进制和， C'_{i+1} 为修正之前的进位

1) 若 $0 \leq X_i + Y_i + C_i < 10$ ，则 $C'_{i+1} = 0$ ， $0 \leq S'_i \leq 9$

$$S_i = S'_i, \quad C_{i+1} = 0$$

2) 若 $10 \leq X_i + Y_i + C_i \leq 15$ ，则 $C'_{i+1} = 0$ ， $10 \leq S'_i \leq 15 \Rightarrow \underline{11xx}$ 或 $\underline{1x1x}$

$$S_i = S'_i + 6 \pmod{16}, \quad C_{i+1} = 1$$

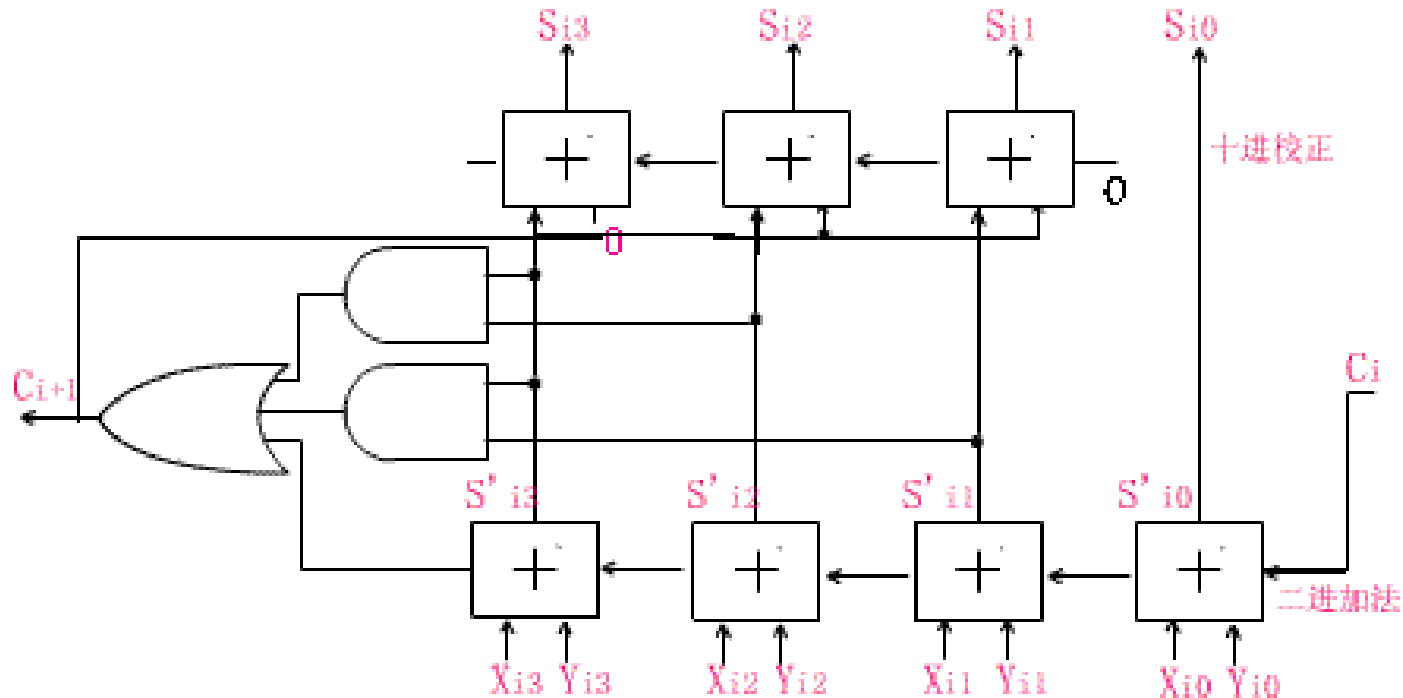
3) 若 $16 \leq X_i + Y_i + C_i \leq 19$ ，则 $C'_{i+1} = 1$ ， $0 \leq S'_i \leq 3$

$$S_i = S'_i + 6 \pmod{16}, \quad C_{i+1} = 1$$



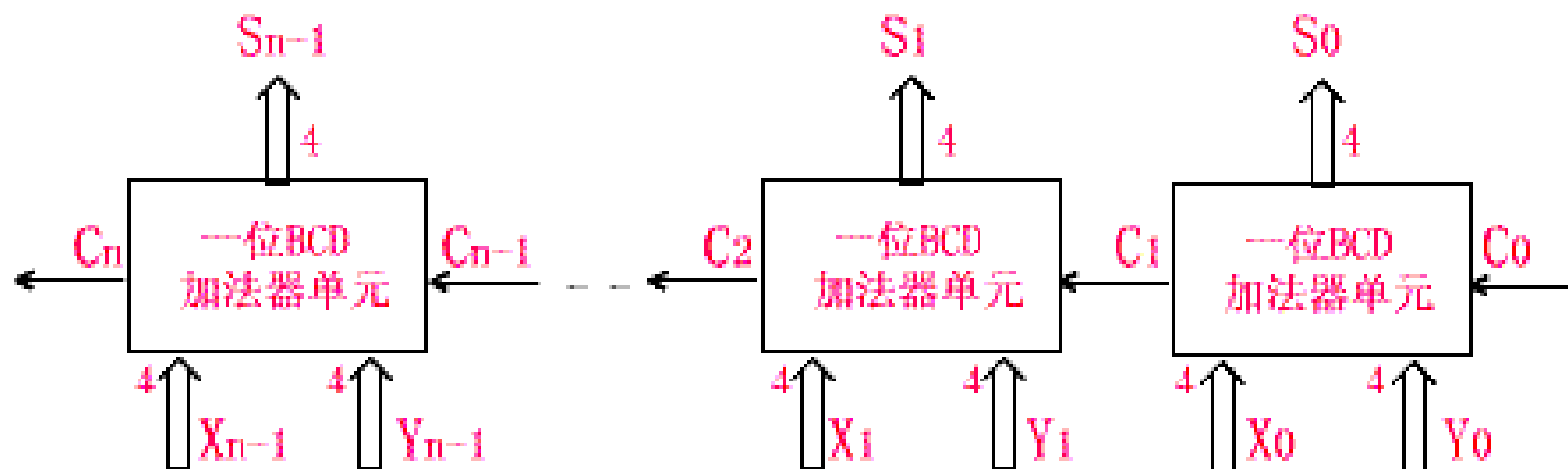


1位十进制加法器电路





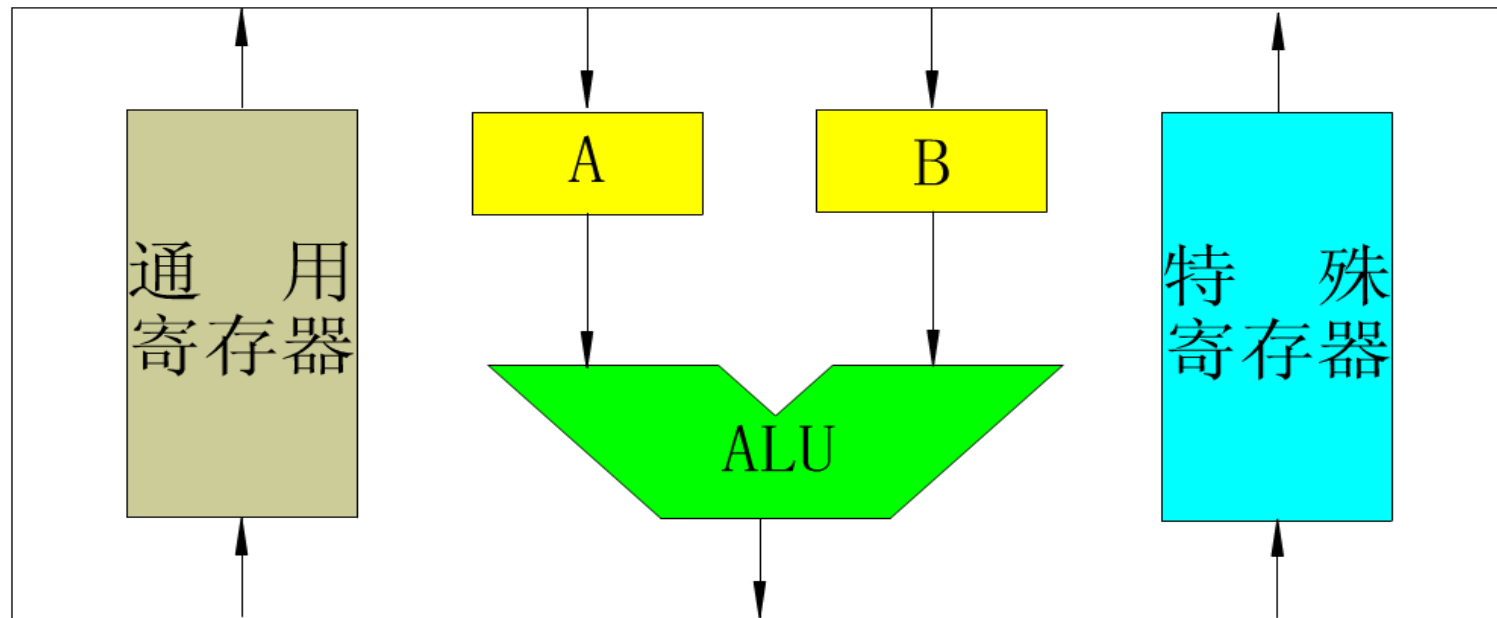
n位数的行波进位BCD加法器





单总线结构运算器

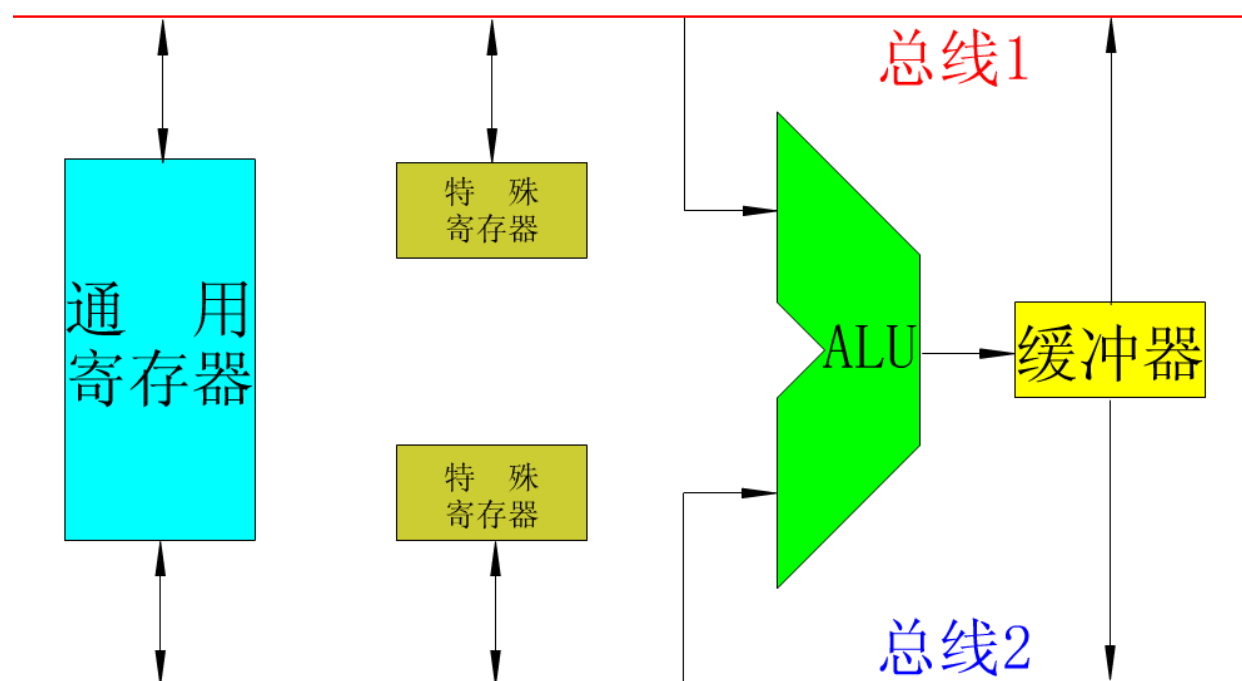
- 所有部件都接到同一总线上
- 在某一时刻，只有一个操作数能放在总线上
- 控制电路比较简单，但操作速度较慢





双总线结构运算器

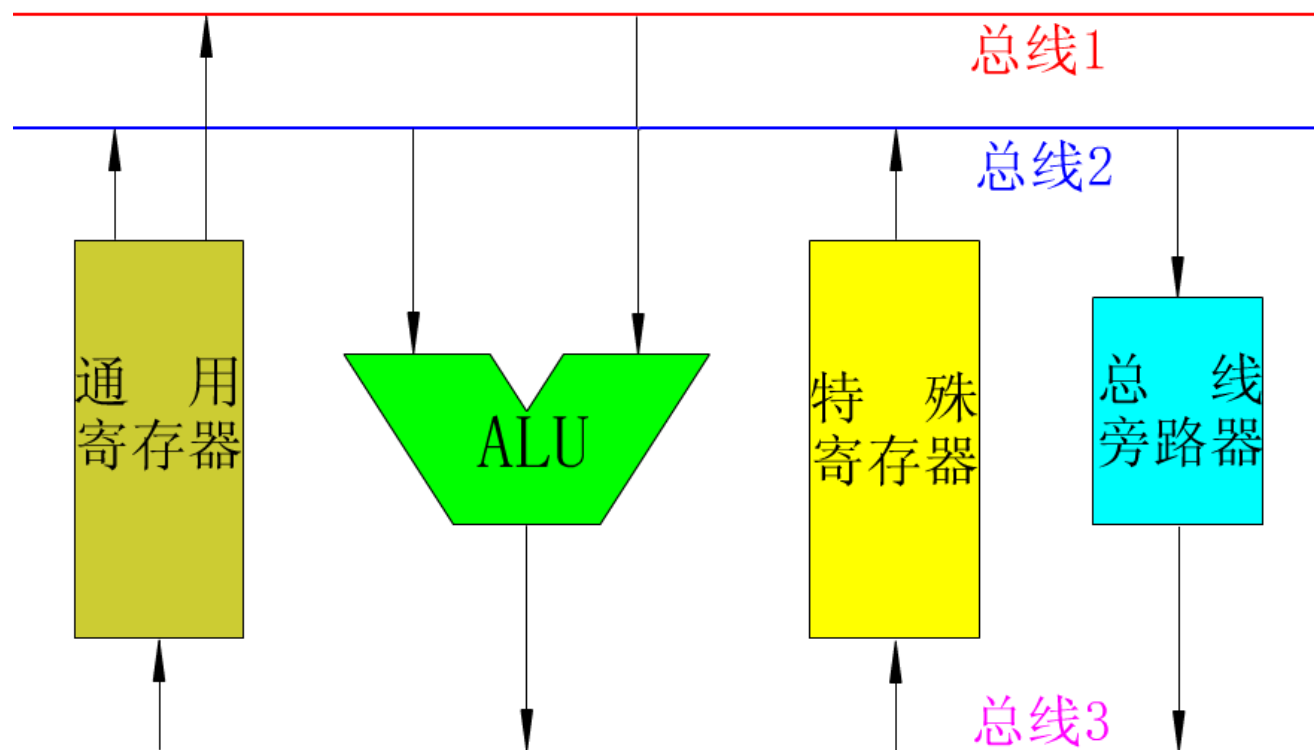
- 两个操作数可同时加到ALU进行运算
- 特殊寄存器分为两组，它们分别与一条总线交换数据
- 控制电路稍复杂，操作速度较快



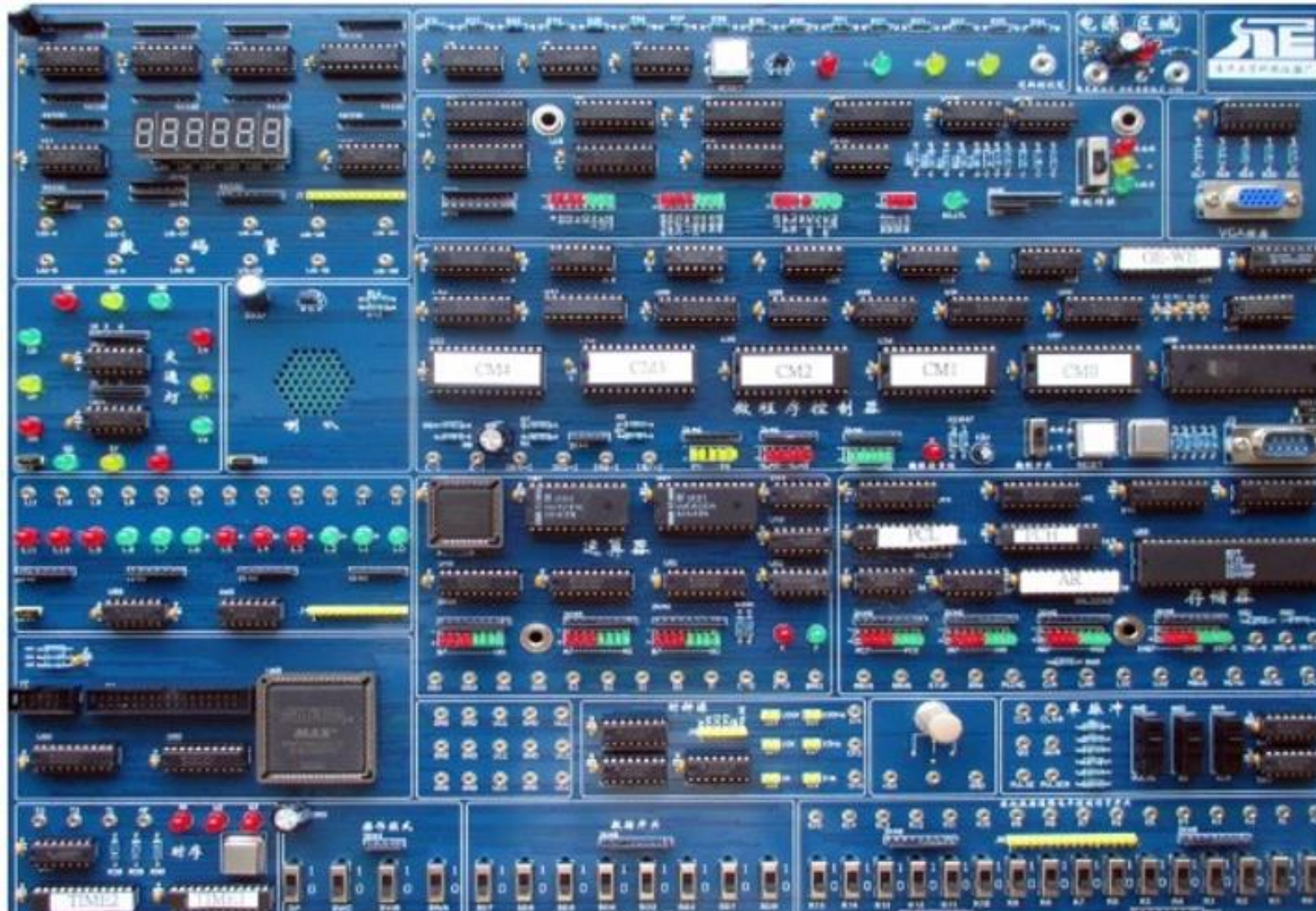


三总线结构运算器

- ALU的两个输入端各连至一条总线，而输出端连至第三条总线。算术逻辑操作可在一步控制之内完成
- 控制电路复杂，操作速度快



实验一 运算器组成



按照实验室老师安排的时间开始做实验